

Manual Técnico y Documentación de Desarrollo: Chrominium

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Semestre 2026-2

Luis Martin Nava Chaparro

Número de Cuenta: 319331718

Temas Selectos III: Realidad Aumentada y Realidad Virtual

Fecha de Entrega: 20 Mayo 2026



Índice

1. Introducción y justificación del proyecto	5
2. Arquitectura y flujo de trabajo técnico	5
3. Análisis de mercado y competencia	6
4. Tecnologías, versiones y requisitos mínimos	6
5. Cronograma de implementación	6
6. Dificultades encontradas y soluciones	7
7. Documentación visual del desarrollo	7
7.1. Configuración inicial del proyecto y Vuforia	7
7.2. Creación del proyecto en Unity Hub	7
7.3. Importación del paquete Vuforia Engine	8
7.4. Despliegue de recursos del paquete	9
7.5. Creación de la cámara de RA	10
7.6. Configuración de licencia de Vuforia	11
7.7. Configuración de Image Targets	13
7.8. Creación de la base de datos en Vuforia	13
7.9. Revisión de la calidad de los marcadores	13
7.10. Descarga de la base de datos	14
7.11. Importación de la base de datos en Unity	15
7.12. Verificación del estado de la base de datos	16
7.13. Importación de modelos 3D	17
7.14. Integración de modelo con su Image Target	17
7.15. Organización de carpetas de materiales y texturas	18
7.16. Importación y previsualización del Alfa Romeo Giulia	19
7.17. Ajustes de importación del modelo	20
7.18. Revisión de materiales del Alfa Romeo	21
7.19. Revisión de texturas PBR del Alfa Romeo	22
7.20. Asignación de mapas de normales	23
7.21. Corrección de materiales interiores con emisión	24
7.22. Iluminación y materiales PBR	25
7.23. Creación del material de skybox	25
7.24. Selección del HDRI para iluminación	26
7.25. Asignación del skybox a la escena	27
7.26. Creación del plano receptor de sombras	28
7.27. Revisión del HDRI en Poly Haven	29
7.28. Escena con skybox por defecto	30
7.29. Aplicación del skybox HDRI	31
7.30. Ajuste de exposición del skybox	32
7.31. Asignación de materiales interiores PBR	33

7.32. Migración a URP y shaders personalizados	34
7.33. Instalación de Universal Render Pipeline	34
7.34. Creación del asset de configuración URP	35
7.35. Validación de URP instalado	36
7.36. Asignación de URP como pipeline global	37
7.37. Conversión de materiales a URP	38
7.38. Resultado de la conversión de materiales	39
7.39. Ajustes avanzados de sombras en URP	41
7.40. Prueba de shadow catcher	42
7.41. Creación del shader ShadowCatcher	43
7.42. Nodos de opacidad para shadow catcher	44
7.43. Validación de escena final	45
7.44. Interfaz de usuario (UI)	46
7.45. Creación del control Slider	46
7.46. Configuración de Canvas y rangos de escala	48
7.47. Validación visual en URP y eventos de Vuforia	48
7.48. Jerarquías de acomodo para escalado uniforme	49
7.49. Configuración de scripts de escala táctil	50
7.50. Reescalado global de nodos Acomodo	52
7.51. Materiales base para cambio de color	52
7.52. Material base del Fiat 500 Abarth	53
7.53. Material base del Alfa Romeo Giulia	54
7.54. Configuración del script de cambio de color	55
7.55. Botón de cambio de color y nuevo escaneo	56
7.56. Edición de texto con TextMeshPro	57
7.57. Fichas técnicas en World Space	58
7.58. Botón de ficha técnica	59
7.59. Catálogos de color por vehículo	60
7.60. Submenú de selección de color	62
7.61. Previsualización de la nueva interfaz	62
7.62. Pantalla de onboarding	63
7.63. Feedback de audio en detección	64
7.64. Color por defecto y validación final	64
7.65. Ajuste final de UI en modo portrait	65
7.66. Verificación de la AR Camera en la jerarquía	67
7.67. Compilación y generación de APK	67
7.68. Configuración básica de Android	67
7.69. Permisos y compatibilidad de RA	68
7.70. Configuración de firma	68
7.71. Generación del APK	68
7.72. Configuración de icono en Android	68
8. Análisis de Costos y Viabilidad	69
9. Métricas de éxito (KPIs)	71

10.Referencias	71
11.Image Targets utilizados	72
12.Consulta y Descarga del repositorio del proyecto en GitHub	74

1. Introducción y justificación del proyecto

Chrominium es una herramienta de Realidad Aumentada (RA) de alta fidelidad orientada a la visualización de vehículos para agencias automotrices. Este manual documenta el proceso técnico de construcción del proyecto en Unity, la integración del SDK de Vuforia, la preparación de activos 3D y la configuración de render e interfaz de usuario, hasta la compilación del APK. El objetivo es que el equipo de desarrollo reproduzca el flujo completo y comprenda el fundamento técnico de cada decisión.

- El problema: el alto costo por metro cuadrado en plazas comerciales y centros de experiencia limita a las agencias a exhibir únicamente uno o dos vehículos físicos.
- La solución: Chrominium funciona como una tecnología de *Pasillo Infinito (Endless Aisle)* que transforma estands pequeños en salas de exhibición virtuales completas.
- Alcance de implementación: se entrega un prototipo funcional en Android que permite visualizar cinco modelos base mediante el escaneo de un marcador visual (*Image Target*).
- Interacción: el usuario puede modificar la pintura, los rines y los interiores de forma instantánea. La aplicación funge como puente para la generación de leads, permitiendo solicitar cotizaciones o pruebas de manejo.

2. Arquitectura y flujo de trabajo técnico

La arquitectura del proyecto se organiza en fases para asegurar trazabilidad de decisiones y control de calidad durante el desarrollo.

- Fase 1: Configuración del Entorno y Motores AR. Creación del proyecto en Unity (Built-In Render Pipeline), importación del SDK de Vuforia Engine y configuración de la base de datos *ProyectoFinal_RA* con marcadores de 5 estrellas de rastreo.
- Fase 2: Importación de Modelos y Materiales PBR. Escalado tridimensional de los modelos (Jeep, Alfa Romeo, Fiat) y configuración de materiales con mapas de Normales y Emisivos (HDR) para emular pintura automotriz y tableros retroiluminados.
- Fase 3: Iluminación Global y Migración a URP. Integración de un mapa HDRI (*Mono-chrome Studio 04*) para reflejos fotorealistas e instalación del Universal Render Pipeline para optimización gráfica móvil.
- Fase 4: Desarrollo de Shaders y Sombras Dinámicas. Programación de un Lit Shader Graph (*ShadowCatcher*) mediante HLSL para lograr el recorte del suelo virtual, dejando únicamente la sombra proyectada.
- Fase 5: Interfaz de Usuario (UI) y Lógica de Interacción. Implementación de scripts de *Pinch Zoom* y controles *Slider* para reescalar objetos, junto con un *InteraccionManager* para alternar colores mediante códigos hexadecimales de fábrica.

- Fase 6: Pulido, Retroalimentación y Compilación. Inserción de UI de onboarding, asignación de retroalimentación auditiva (*AR Feedback*) y configuración de íconos adaptativos para la compilación del APK final.

3. Análisis de mercado y competencia

- Mercado objetivo: agencias automotrices que buscan presencia en plazas de alto tráfico reduciendo gastos operativos.
- Competencia: showrooms físicos tradicionales (alto costo) y visualizadores 3D web genéricos (baja fidelidad y sin integración con entorno real).
- Diferenciador: fotorrealismo en móviles mediante URP, materiales PBR y oclusión de sombras dinámicas con *ShadowCatcher* en un entorno AR.

4. Tecnologías, versiones y requisitos mínimos

- Motor gráfico: Unity 6000.0.66f1 (LTS).
- SDK de RA: Vuforia Engine 11.4.4.
- Motor de renderizado: Universal Render Pipeline (URP) 17.0.4.
- Modelado y ajustes 3D: Autodesk Maya 2024 / Blender.
- Lenguaje: C# (scripts de interacción UI, reescalado y cambio de color).
- Requisitos mínimos del dispositivo: Android 10 o superior, 4 GB de RAM, cámara trasera de al menos 12 MP compatible con ARCore, giroscopio y acelerómetro.

5. Cronograma de implementación

- Fase 1 (Semanas 1-2): planeación UI/UX, investigación de mercado y adquisición de modelos 3D optimizados (SketchFab).
- Fase 2 (Semanas 3-4): configuración del proyecto en Unity, alta en Vuforia Target Manager y evaluación de Image Targets (5 estrellas).
- Fase 3 (Semanas 5-7): asignación de texturas PBR, corrección de normales, mapas emisivos y configuración de iluminación global HDRI (Poly Haven).
- Fase 4 (Semanas 8-9): migración a URP y desarrollo de nodos en Shader Graph para *ShadowCatcher*.
- Fase 5 (Semanas 10-11): programación C# (Control Slider, Pinch Zoom, AR Persistencia) y diseño de la UI (cánvases en *World Space*).
- Fase 6 (Semana 12): pruebas de rendimiento, compilación de APK y redacción de documentación técnica.

6. Dificultades encontradas y soluciones

- Dificultad 1: las sombras de los vehículos no se proyectaban sobre el entorno real, rompiendo la inmersión.
- Solución: desarrollo de un *Lit Shader Graph* personalizado con funciones HLSL que captura la atenuación de la luz direccional y recorta la opacidad del plano base (*ShadowCatcher*), dejando solo la sombra.
- Dificultad 2: pérdida de tracking del marcador bajo iluminación intensa en plazas.
- Solución: habilitación de *Continuous Autofocus* en la AR Camera y rediseño de los Image Targets para alto contraste (rating de 5 estrellas en Vuforia).
- Dificultad 3: renderizado ineficiente de interiores saturando la GPU del móvil.
- Solución: uso de texturas *DiffuseAOSO* y *Baked Emission* para simular iluminación en tableros sin luces en tiempo real adicionales.

7. Documentación visual del desarrollo

7.1. Configuración inicial del proyecto y Vuforia

En esta sección se documenta el arranque del proyecto en Unity y la integración base del SDK de Vuforia. Se parte de una plantilla 3D convencional para asegurar compatibilidad con los paquetes de RA y se configura la licencia que habilita el motor de rastreo en dispositivo.

7.2. Creación del proyecto en Unity Hub

Se crea un proyecto limpio para garantizar una jerarquía de escenas sin dependencias previas. El pipeline Built-In proporciona una base estable para validar el rastreo y la interacción antes de migrar a un pipeline más avanzado.

- Creación de un nuevo proyecto en Unity Hub.
 - Se selecciona la plantilla *3D (Built-In Render Pipeline)*.
 - Se define el nombre del proyecto y la ubicación de almacenamiento local.

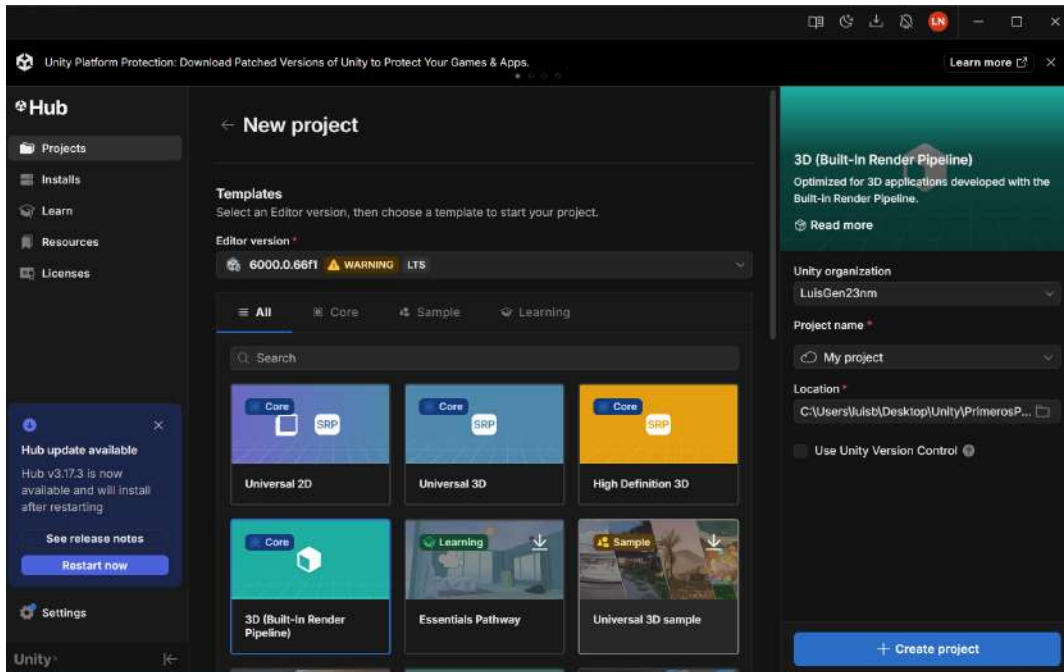


Figura 1: Creación del proyecto base en Unity Hub.

7.3. Importación del paquete Vuforia Engine

Vuforia se integra desde el Package Manager para mantener el SDK versionado y consistente con el proyecto. Este enfoque facilita la resolución de dependencias y evita instalaciones manuales que pueden afectar la compatibilidad con el editor.

- Importación del paquete Vuforia Engine en Unity.
 - Se muestra la ventana del Package Manager advirtiéndolo sobre las dependencias requeridas.
 - El sistema prepara la instalación de los módulos básicos para el funcionamiento del SDK de Realidad Aumentada.

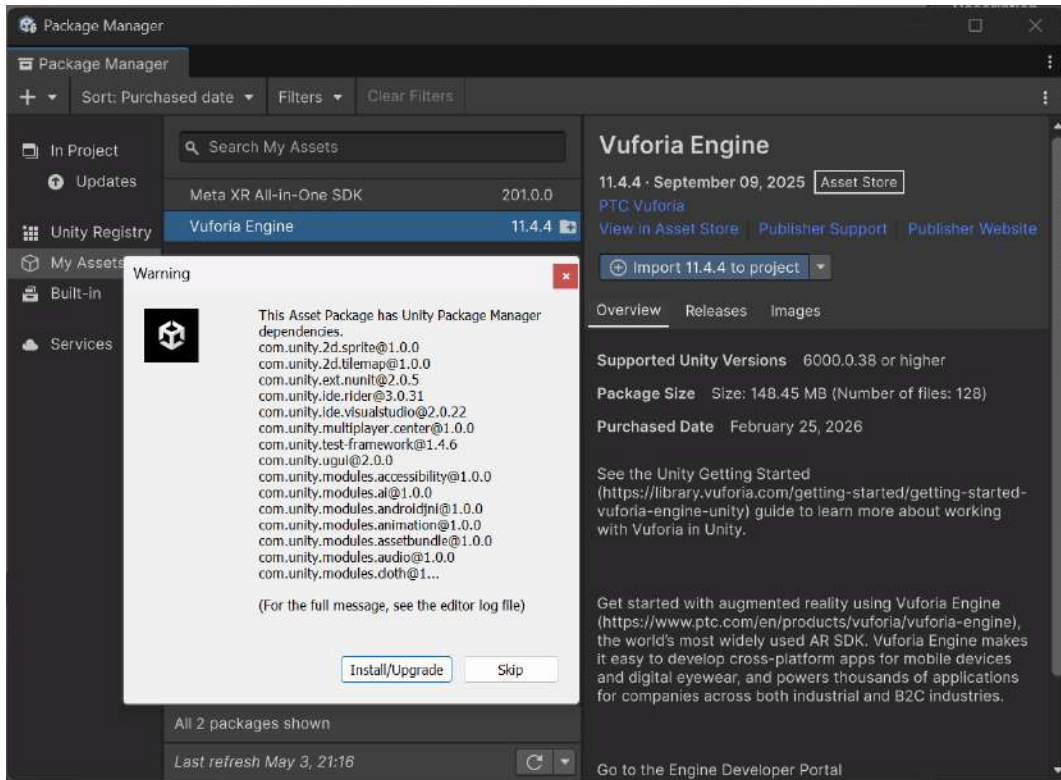


Figura 2: Instalación de Vuforia desde el Package Manager.

7.4. Despliegue de recursos del paquete

La importación masiva del paquete registra los prefabs, scripts y assets necesarios para la RA. En esta fase se validan carpetas específicas, como iconos adaptativos, que se utilizarán en el proceso de build móvil.

- Despliegue de los componentes del paquete Vuforia Engine.
 - La ventana *Import Unity Package* lista todos los recursos a instalar.
 - Se destacan las carpetas de iconos adaptativos para sistemas Android e iOS.

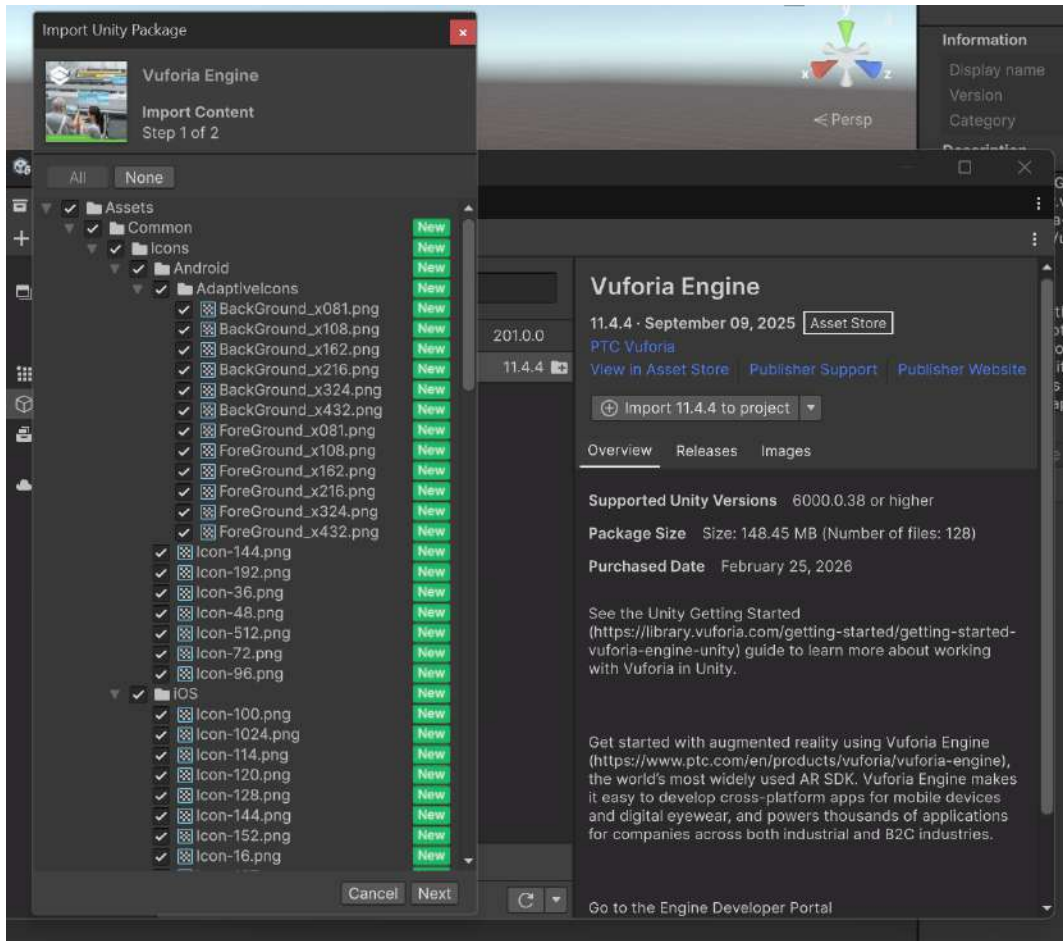


Figura 3: Importación de recursos del paquete Vuforia.

7.5. Creación de la cámara de RA

La cámara de Vuforia sustituye la cámara estándar para inyectar el pipeline de tracking y la calibración necesaria para el reconocimiento de Image Targets. Esto asegura que el render de los modelos 3D quede alineado con el entorno real.

- Creación de la cámara de Realidad Aumentada en la escena.
 - Mediante el menú contextual en la Jerarquía (Hierarchy), se navega a *Vuforia Engine*.
 - Se selecciona *AR Camera* para reemplazar la cámara estándar del proyecto.

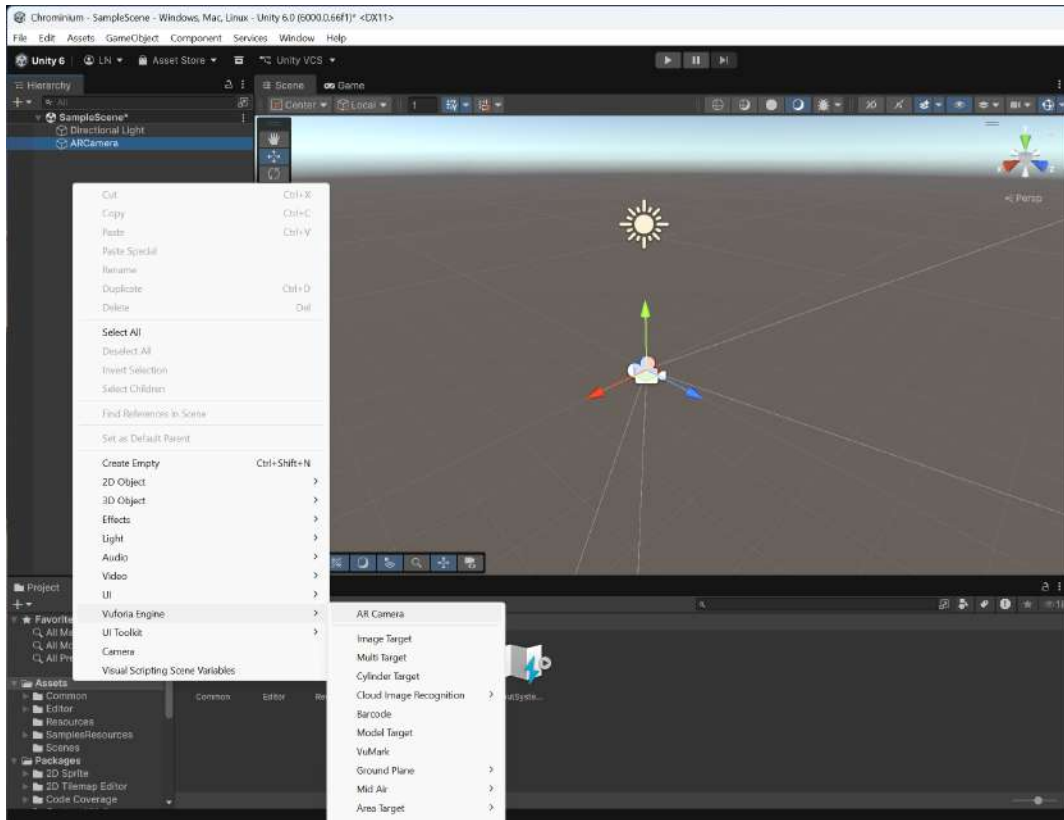


Figura 4: Inserción de la AR Camera de Vuforia.

7.6. Configuración de licencia de Vuforia

La licencia habilita el motor de tracking en builds reales. Sin este paso, la aplicación puede ejecutarse en el editor pero no reconocer objetivos en dispositivo; por ello se valida desde el inicio.

- Configuración global y validación de licencia de Vuforia.
 - En el panel del Inspector se despliega la *Vuforia Configuration*.
 - Se ingresa la clave alfanumérica en *App License Key* para habilitar las funcionalidades de rastreo en el dispositivo.

7.7. Configuración de Image Targets

Los Image Targets constituyen el mecanismo de anclaje visual que permite superponer los modelos 3D sobre logotipos físicos. Se utiliza una base de datos local tipo *Device* para garantizar disponibilidad offline durante demostraciones en agencias.

7.8. Creación de la base de datos en Vuforia

La base de datos centraliza los marcadores que la app reconocerá. Elegir el tipo *Device* permite que los targets se empaqueten directamente en el APK y no dependan de servicios en la nube.

- Creación de una base de datos en el portal de desarrolladores de Vuforia.
 - Se utiliza el Target Manager en el navegador web para generar una nueva base de datos.
 - La base se nombra *ProyectoFinal_RA* y se clasifica con el tipo *Device* para almacenamiento local.

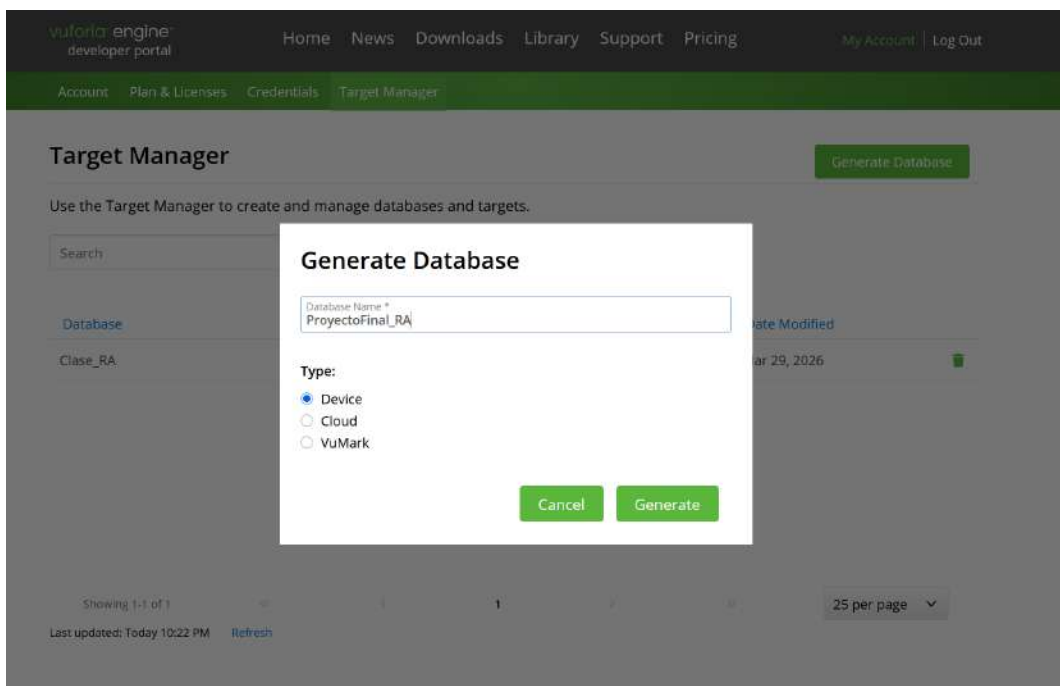


Figura 6: Creación de la base de datos de Image Targets.

7.9. Revisión de la calidad de los marcadores

Antes de descargar, se verifica la calidad de tracking para minimizar falsos negativos en condiciones de iluminación variables. Una calificación de 5 estrellas asegura estabilidad de reconocimiento.

- Revisión de los marcadores de imagen (Image Targets) en la base de datos.
 - Se listan los 4 marcadores correspondientes a los logotipos de los vehículos.
 - Se verifica que todos cuentan con una calificación óptima de rastreo de 5 estrellas (*Rating*).

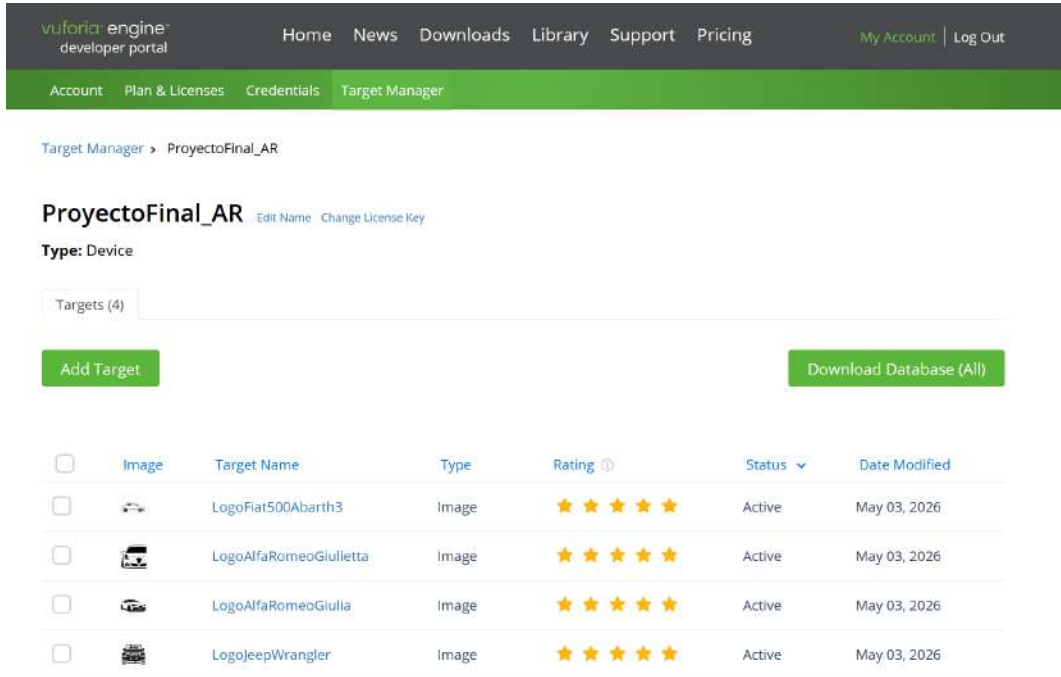


Figura 7: Validación de la calidad de los Image Targets.

7.10. Descarga de la base de datos

La exportación en formato de Unity permite integrar los targets como un paquete nativo, manteniendo texturas y metadatos alineados con el SDK.

- Descarga de la base de datos de marcadores desde el portal.
 - Se selecciona la opción *Download Database*.
 - Se elige *Unity Editor* como plataforma de desarrollo para exportar el paquete de datos compatible.

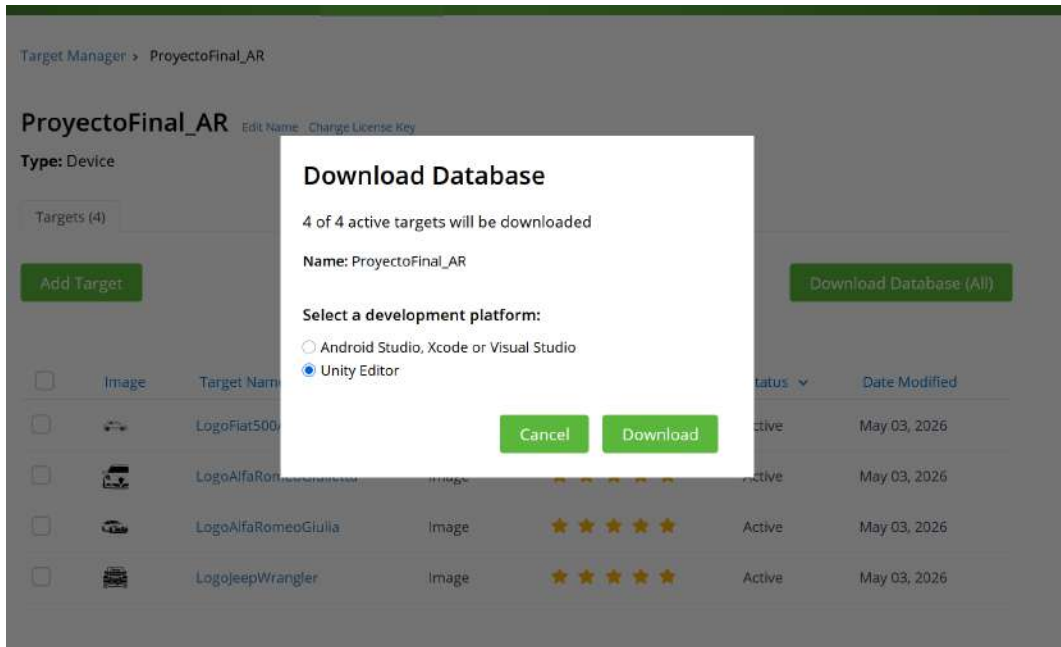


Figura 8: Descarga del paquete de base de datos para Unity.

7.11. Importación de la base de datos en Unity

Se integra el paquete para que Vuforia detecte los targets en el editor y permita asignarlos en escena. Este paso habilita el flujo de trabajo de arrastre y configuración de Image Targets.

- Importación de la base de datos de Vuforia a Unity.
 - La ventana *Import Unity Package* muestra las texturas escaladas listas para integrarse.
 - Se observa la sincronización con el explorador de archivos de Windows para ubicar el paquete *.unitypackage*.

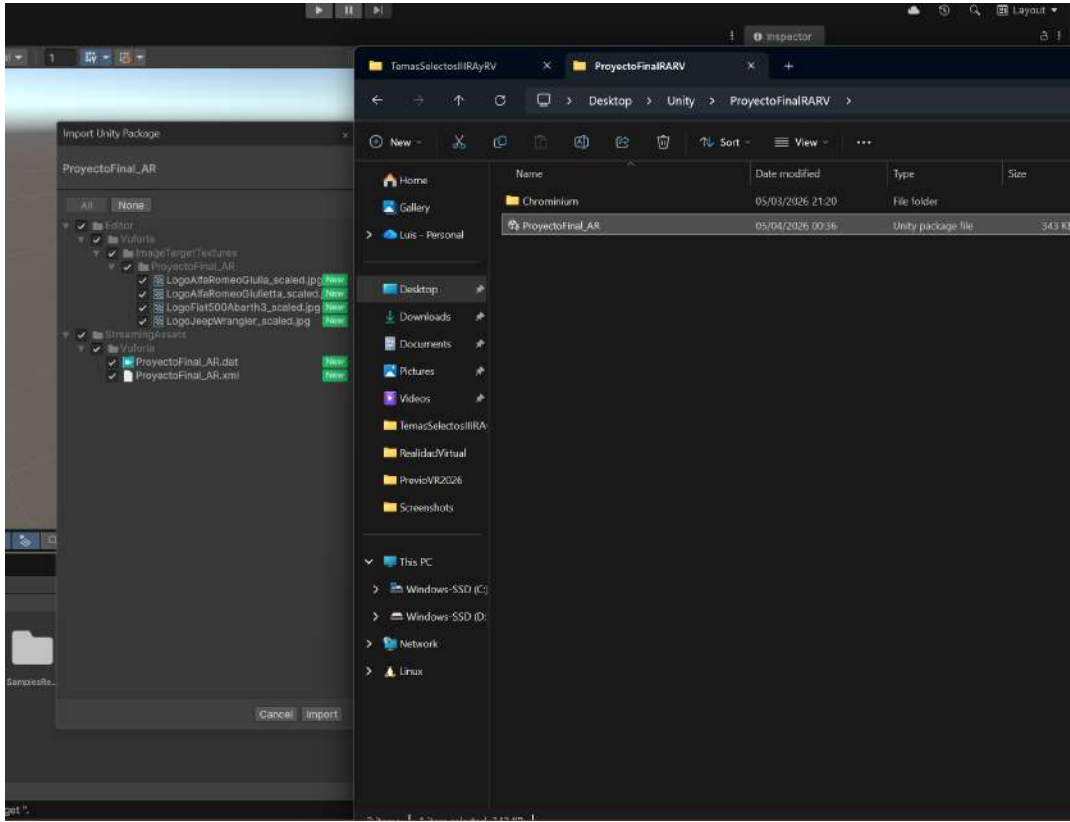


Figura 9: Importación de la base de datos de targets en Unity.

7.12. Verificación del estado de la base de datos

Se valida en el portal que los targets estén activos para evitar discrepancias entre la base en la nube y el paquete importado.

- Verificación del estado de los marcadores en Vuforia.
 - Vista general del Target Manager en la plataforma web.
 - Confirma que la base de datos *ProjectoFinal_AR* tiene sus 4 Image Targets activos y listos para uso.

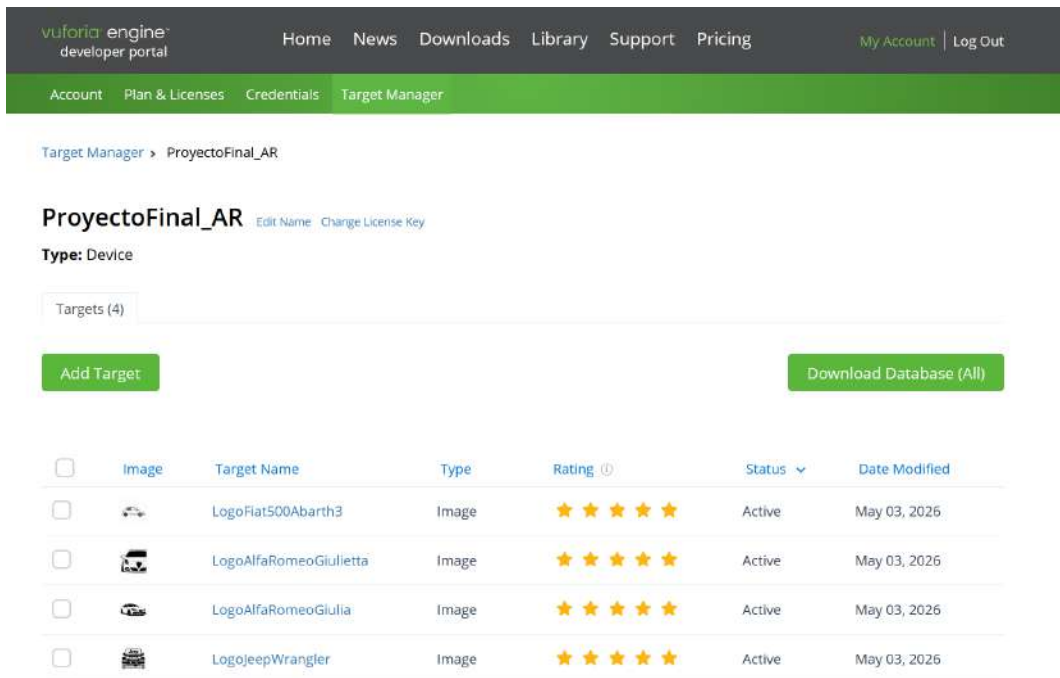


Figura 10: Estado de la base de datos y targets activos.

7.13. Importación de modelos 3D

La integración de modelos 3D se realiza junto con sus Image Targets para validar escala, orientación y fidelidad visual desde el inicio. Este enfoque reduce retrabajo durante ajustes de iluminación o materiales.

7.14. Integración de modelo con su Image Target

Se coloca el modelo sobre su marcador correspondiente para asegurar que la proyección en RA coincida con el tamaño esperado en el entorno físico.

- Integración del modelo 3D y su marcador en la escena.
 - Se visualiza el modelo 3D del Jeep Wrangler Rubicon posicionado sobre su respectivo marcador (Image Target).
 - El vehículo está escalado y alineado tridimensionalmente para su correcta proyección en Realidad Aumentada.

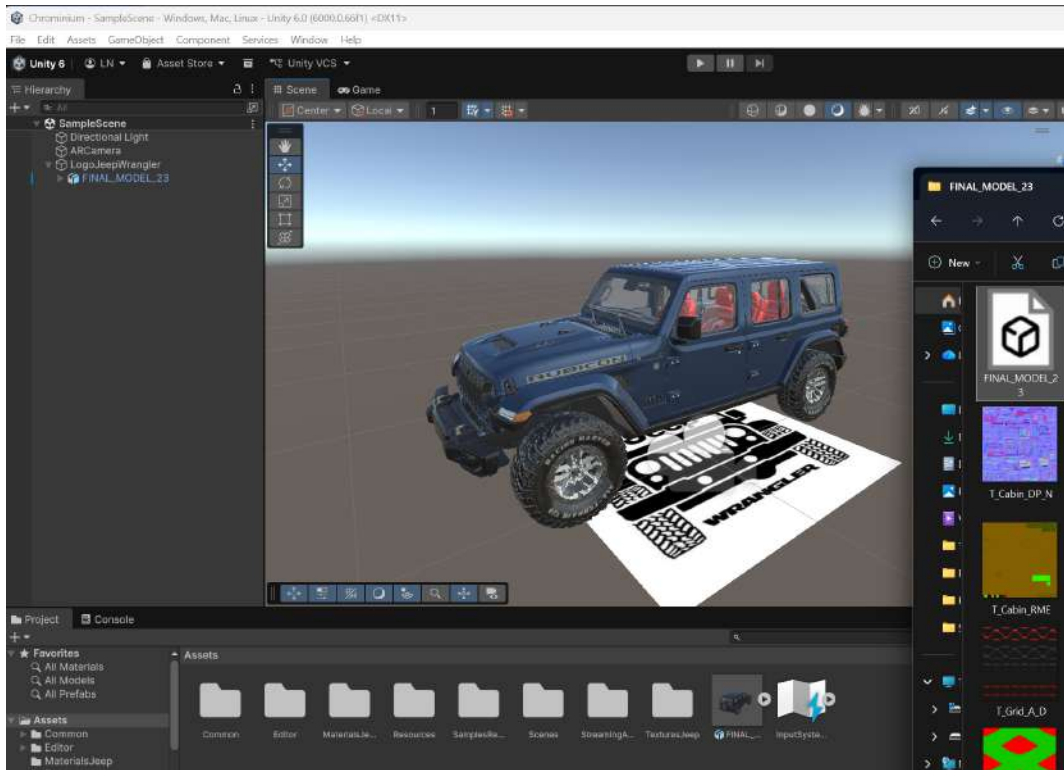


Figura 11: Modelo 3D alineado con su Image Target.

7.15. Organización de carpetas de materiales y texturas

Una estructura de carpetas clara simplifica el mantenimiento de materiales PBR, reduce conflictos de referencias y facilita la migración a nuevos pipelines.

- Organización de la estructura de carpetas del proyecto.
 - En la ventana *Select Textures Folder*, se crean y organizan los directorios dentro de *Assets*.
 - Se destaca la carpeta *TexturesGiulietta* para agrupar ordenadamente los materiales del Alfa Romeo.

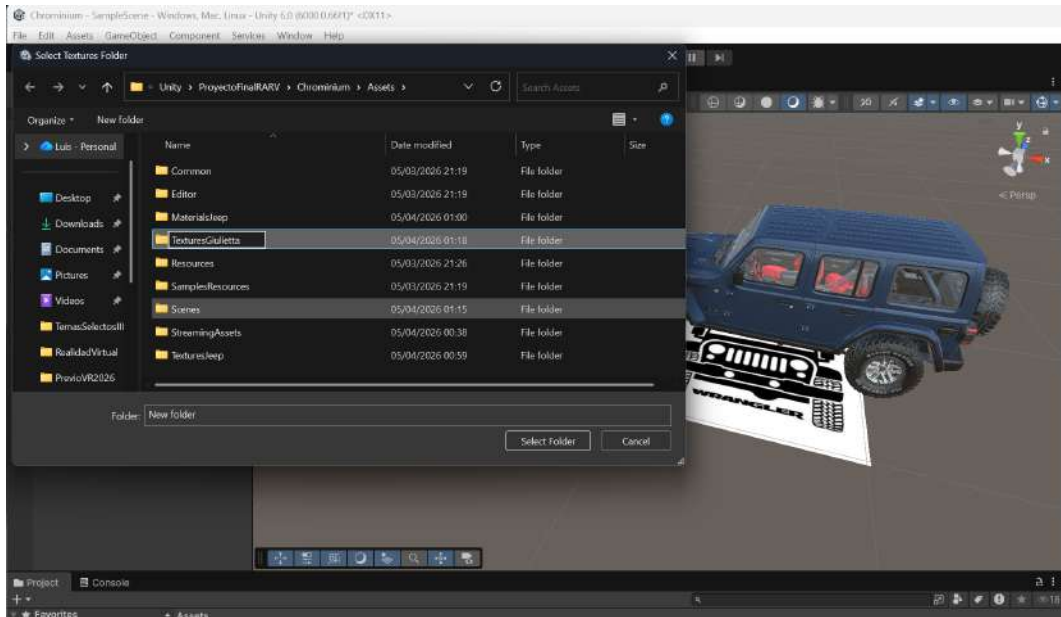


Figura 12: Estructura organizada de carpetas para texturas.

7.16. Importación y previsualización del Alfa Romeo Giulia

Se revisa el modelo en escena para detectar problemas de malla y materiales. Las advertencias de normales y tangentes anticipan ajustes de iluminación y sombreado necesarios para mantener la fidelidad visual.

- Importación y previsualización del modelo 3D *Alfa Romeo Giulia* en la escena de Unity.
 - En la Jerarquía, el objeto *FINAL_MODEL_QU* se posiciona bajo su respectivo marcador *LogoAlfaRomeoGiulia*.
 - La consola muestra múltiples advertencias (*Warnings*) de importación relacionadas con el cálculo de normales y tangentes de la malla del modelo.
 - El Inspector muestra un material temporal *Opaque* aplicado al vehículo.

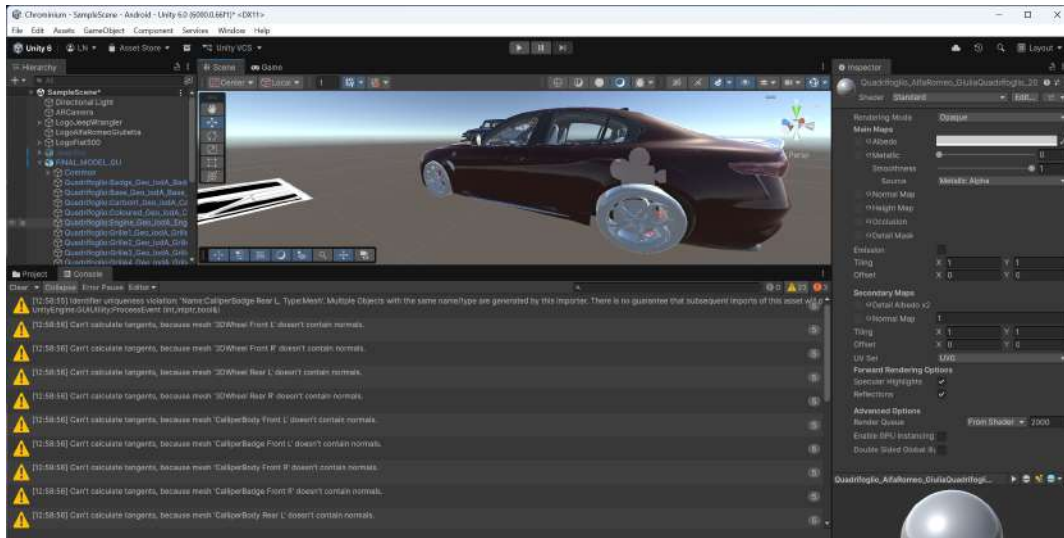


Figura 13: Modelo Giulia importado con advertencias de malla.

7.17. Ajustes de importación del modelo

El factor de escala se calibra para que el tamaño en RA coincida con proporciones reales. El cálculo de normales mejora la lectura de superficies en iluminación PBR.

- Panel de configuración de importación de modelo (*Model Import Settings*) para el archivo *FINAL_MODEL_QU.fbx* del Alfa Romeo.
 - La pestaña *Model* está activa en el Inspector.
 - Se configura el *Scale Factor* (Factor de Escala) para ajustar el tamaño del modelo real al entorno de Realidad Aumentada.
 - Se observa la activación de opciones de optimización de malla y cálculo de normales (*Normals Mode*).



Figura 14: Configuración de importación del modelo y escala.

7.18. Revisión de materiales del Alfa Romeo

Se inspeccionan los materiales exportados para asegurar que cada componente tenga su *.mat* correspondiente, lo que facilita reasignar propiedades físicas y mapas PBR.

- Explorador de archivos de Windows (*File Explorer*) mostrando el contenido de la carpeta *GiuliaMaterials*.
 - Se listan múltiples archivos con extensión *.mat* (Materiales de Unity) y sus respectivos archivos de metadatos (*.mat.meta*).
 - Estos materiales controlan las propiedades físicas de la pintura, cristales, rines y componentes del motor del modelo Alfa Romeo.

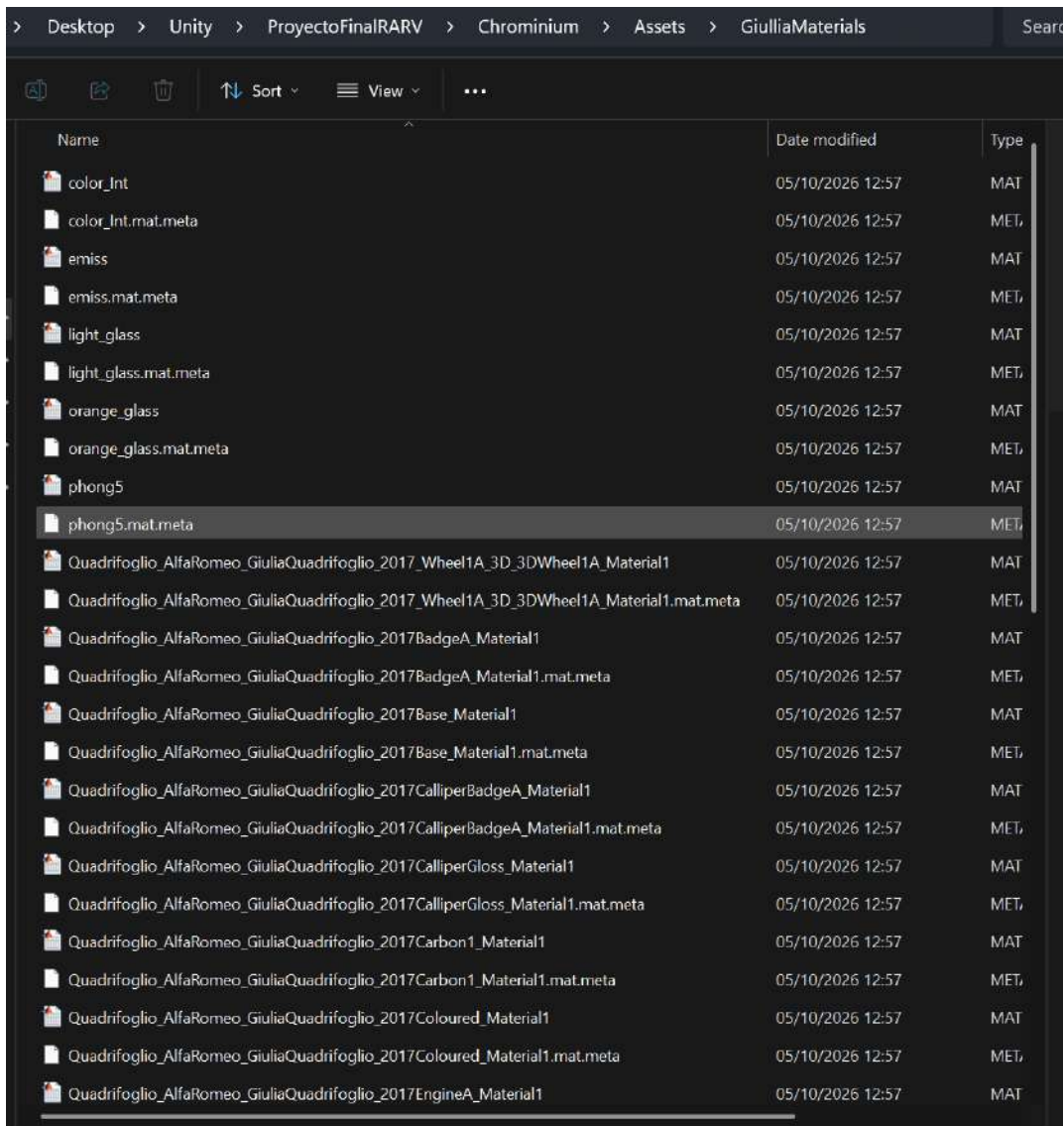


Figura 15: Materiales exportados del Alfa Romeo Giulia.

7.19. Revisión de texturas PBR del Alfa Romeo

Se identifican mapas de normales, difusos y emisivos para asegurar consistencia en el shading. Esto permite un mapeo directo con el shader *Standard* o con URP posteriormente.

- Explorador de archivos de Windows mostrando el contenido de la carpeta *GiuliaTextures*.
 - Se listan archivos de imagen (PNG) que contienen las texturas 2D del vehículo.
 - Se identifican mapas de Normales (*Normal*), colores difusos (*DiffuseAOSO*) y emisivos (*Emissive*) listos para ser importados a Unity.

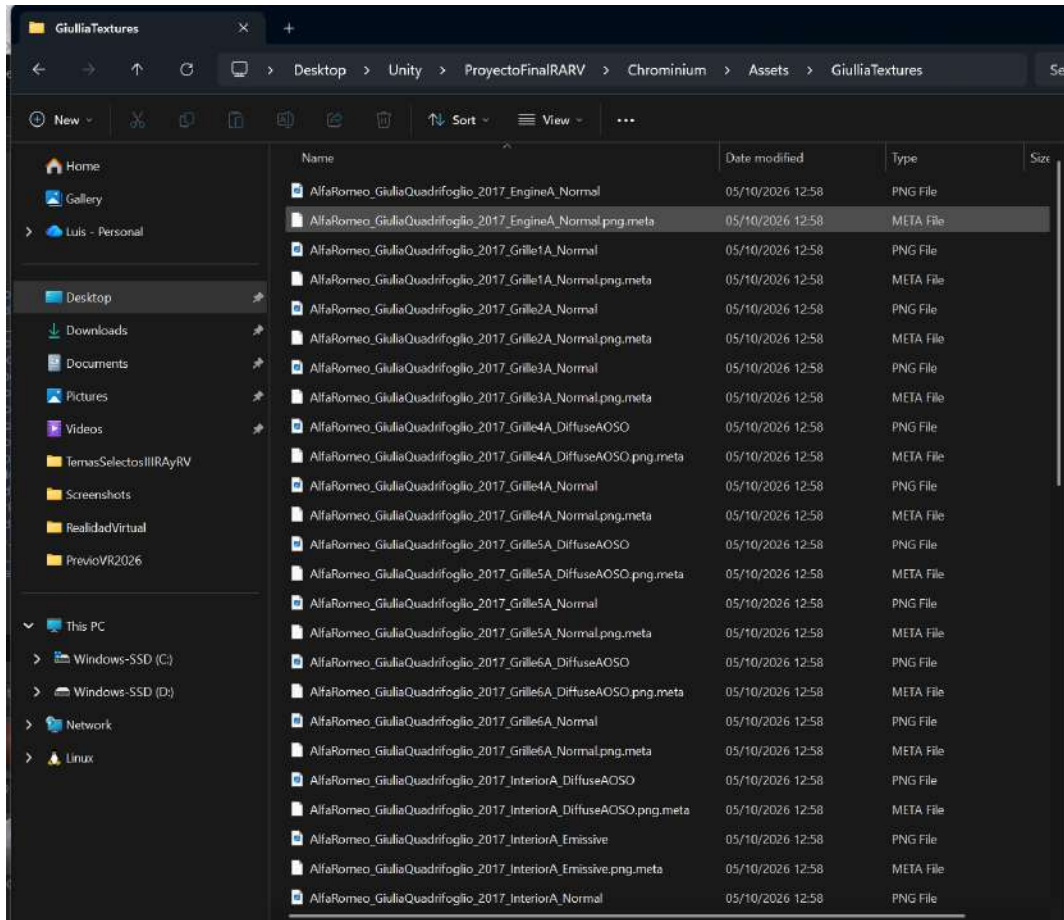


Figura 16: Texturas PBR del modelo Giulia.

7.20. Asignación de mapas de normales

El normal map incrementa el detalle aparente sin elevar el conteo de poligonos, lo cual es clave para rendimiento en dispositivos móviles.

- Aplicación de un mapa de normales (*Normal Map*) al material de los rines del Alfa Romeo en Unity.
 - En el panel inferior de *Assets*, se seleccionan las texturas color violeta claro características de los mapas de normales.
 - En el Inspector, se asigna la textura al canal *Normal Map* del shader.
 - La vista de escena refleja el aumento de detalle y relieve en la geometría del rin sin necesidad de aumentar la carga poligonal.



Figura 17: Normal map aplicado a rines para mayor detalle.

7.21. Corrección de materiales interiores con emisión

La emisión controla la retroiluminación de indicadores, mejorando legibilidad del tablero sin depender de luces externas.

- Corrección y ajuste fino de los materiales interiores del habitáculo del Alfa Romeo Giulia.
 - La cámara de la Escena realiza un acercamiento (*zoom*) a los instrumentos del tablero.
 - Se configura un material de tipo emisor (*Emission*) en el Inspector, marcando la casilla de color blanco HDR.
 - Esto simulará la retroiluminación de los indicadores analógicos y pantallas digitales dentro del modelo 3D.



Figura 18: Materiales interiores con emisión HDR.

7.22. Iluminación y materiales PBR

El realismo visual en una aplicación automotriz depende de reflejos consistentes y sombras creíbles. Por ello se configura un skybox HDRI para iluminación global y un plano receptor de sombras para anclar los modelos al entorno.

7.23. Creación del material de skybox

Un material de skybox con cubemap permite simular un entorno 360° y generar reflejos coherentes en superficies metálicas.

- Creación del material para el entorno de iluminación (Skybox).
 - En el panel del Inspector, se configura el material *Skybox_Studio_Material*.
 - Se le asigna el Shader específico *Skybox/Cubemap* para proyectar imágenes en 360 grados.

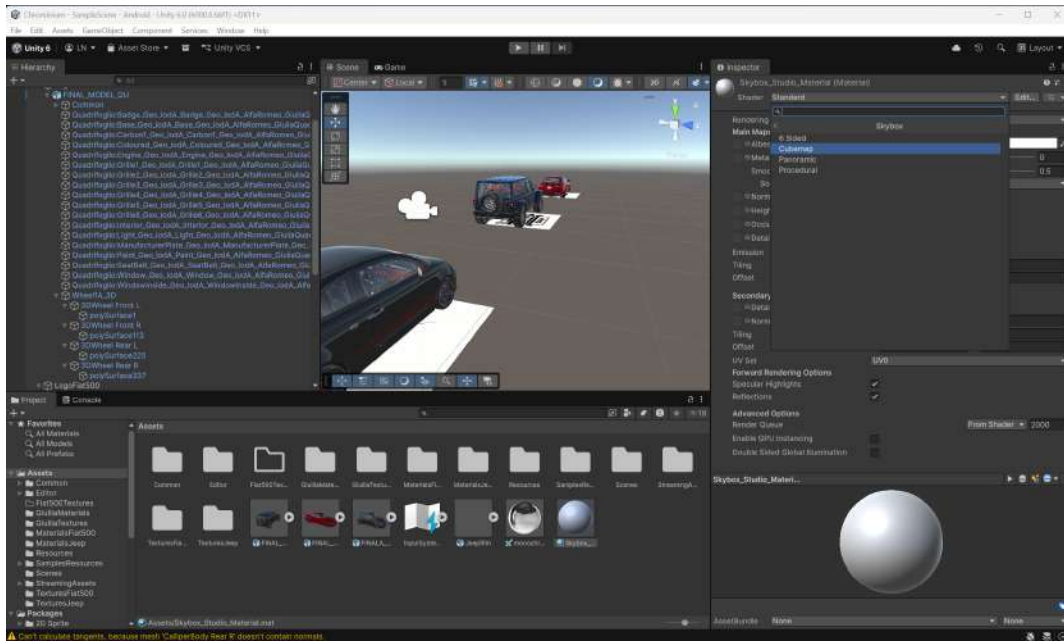


Figura 19: Material de skybox configurado con cubemap.

7.24. Selección del HDRI para iluminación

Se elige un HDRI de estudio automotriz para simular un set de exhibición con reflejos suaves y controlados, mejorando la lectura de la carrocería.

- Descarga de recursos de iluminación desde Poly Haven.
 - En el navegador web se muestra la ficha del HDRI *Monochrome Studio 04*.
 - Este mapa ambiental gratuito se utilizará para simular la iluminación y reflejos de un entorno de exhibición automotriz.

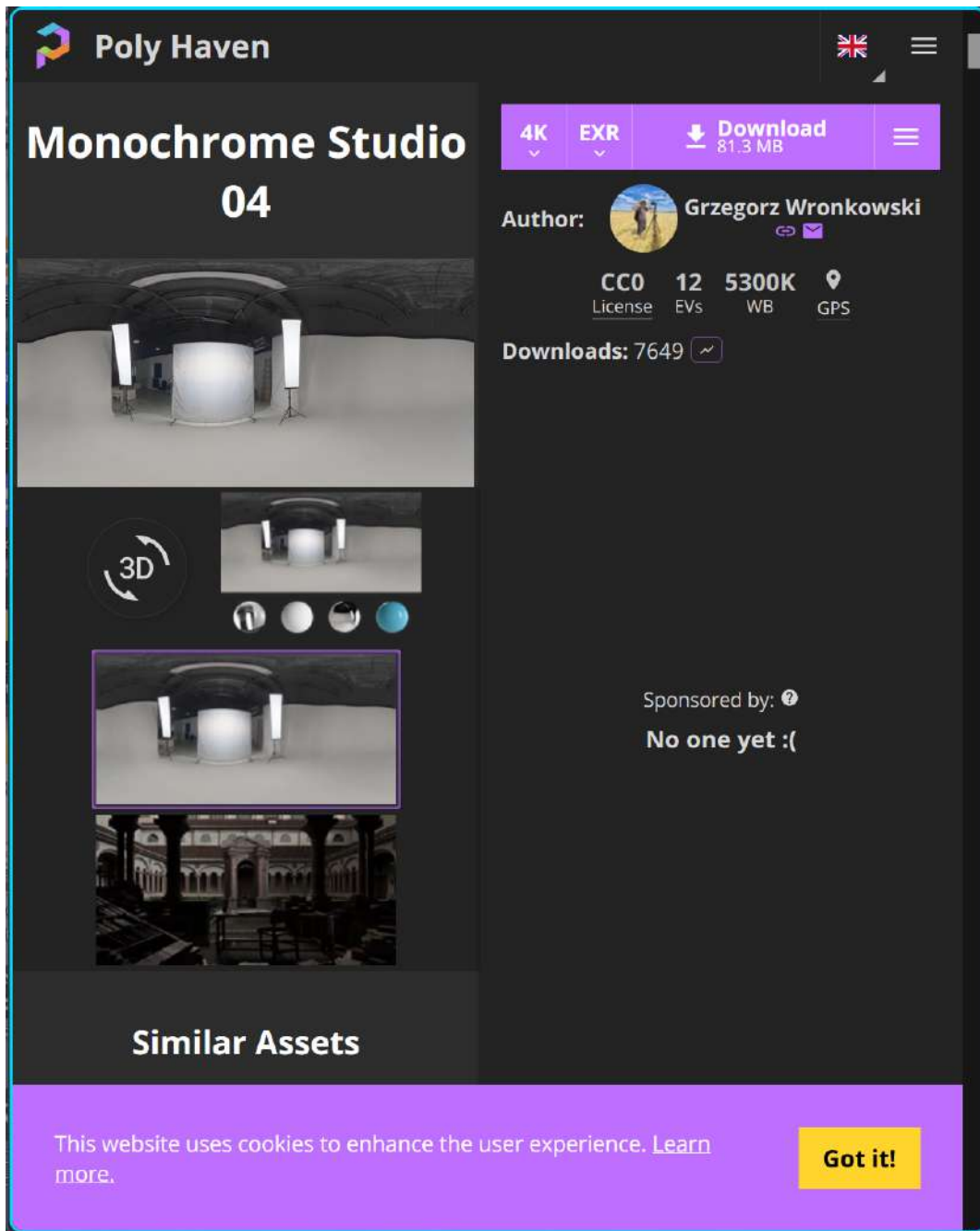


Figura 20: HDRI seleccionado para iluminación de estudio.

7.25. Asignación del skybox a la escena

Al asignar el skybox en *Lighting*, se actualizan de inmediato los reflejos y el aporte de luz indirecta en los materiales PBR.

- Asignación del Skybox a la iluminación global de la escena.
 - En la ventana de configuración *Lighting*, pestaña *Environment*, se asigna el mate-

rial *Skybox_Studio_Material*.

- El modelo del Alfa Romeo reacciona inmediatamente mostrando reflejos metálicos realistas sobre su pintura.



Figura 21: Aplicación del skybox y respuesta de reflejos.

7.26. Creación del plano receptor de sombras

Un plano receptor permite anclar visualmente el modelo al entorno real, evitando la percepción de que el vehículo flota sobre el marcador.

- Creación del plano receptor de sombras.
 - En la Jerarquía se observa un objeto 3D tipo *Plane* nombrado *Suelo_Sombras*.
 - El plano se posiciona estratégicamente debajo de las ruedas del Jeep para recibir las sombras proyectadas.

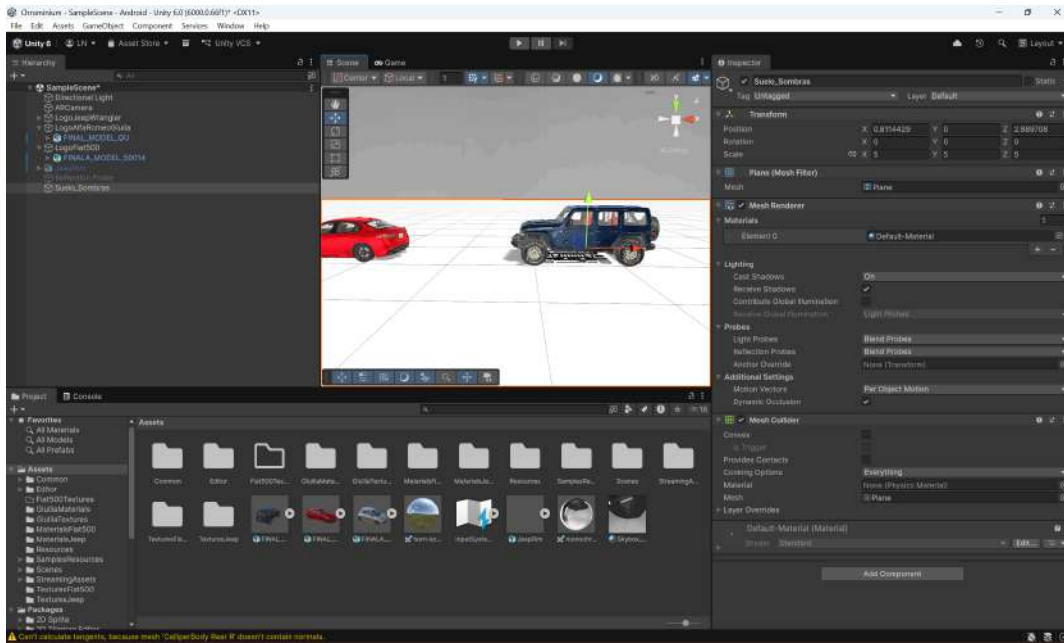


Figura 22: Plano receptor de sombras para anclaje visual.

7.27. Revisión del HDRI en Poly Haven

Se revalida el recurso HDRI para asegurar consistencia de iluminación y mantener el mismo entorno de estudio durante ajustes posteriores.

- Reingreso a la plataforma web Poly Haven para la revisión del recurso HDRI *Monochrome Studio 04*.
 - Se muestra la interfaz de descarga gratuita del mapa ambiental en 360 grados.
 - Este recurso se utilizará para generar reflejos y destellos de luz controlados, simulando un entorno de exhibición automotriz.

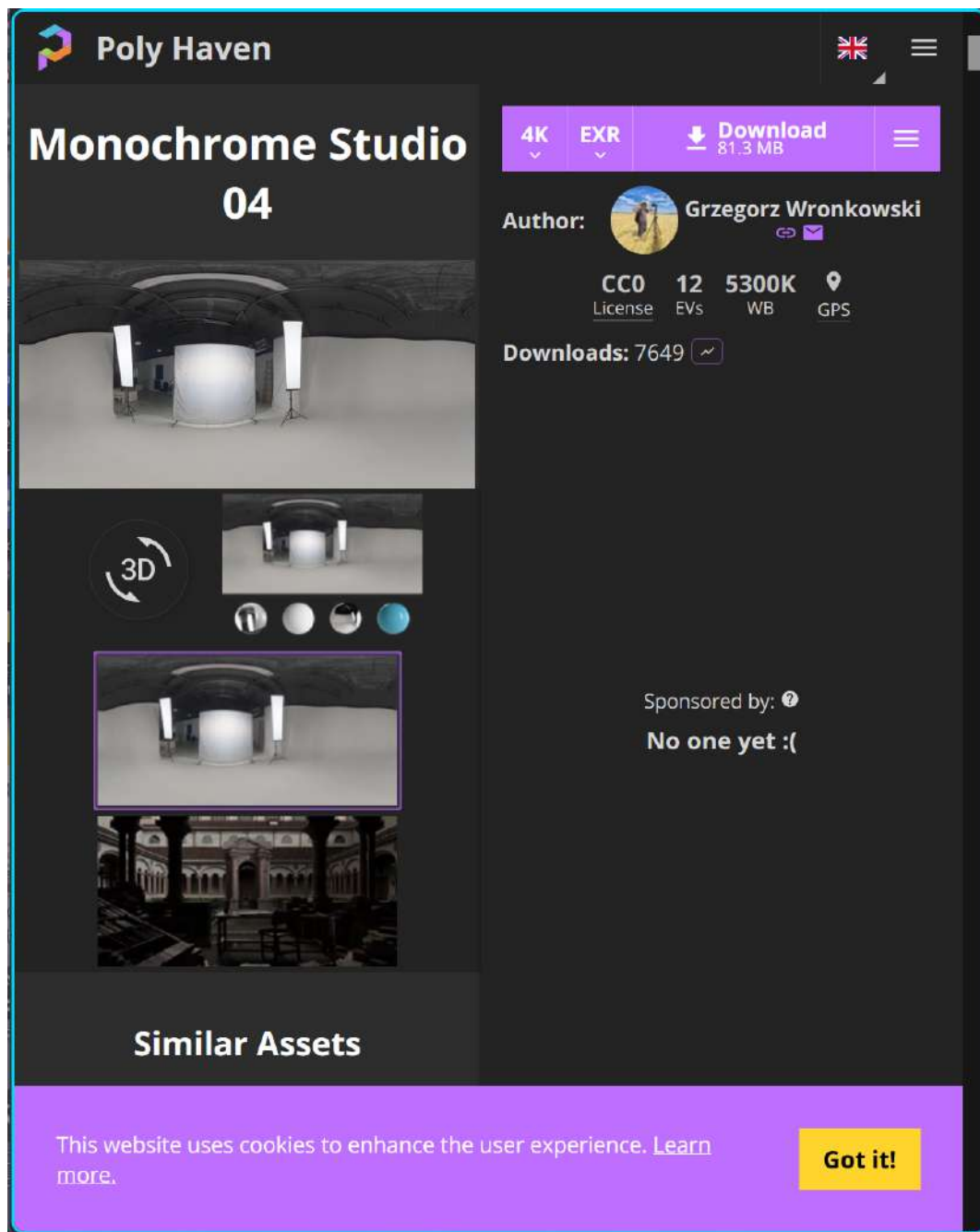


Figura 23: Revisión del HDRI en Poly Haven.

7.28. Escena con skybox por defecto

Se identifica la iluminación plana del *Default-Skybox* como referencia antes de aplicar el HDRI de estudio.

- Visualización panorámica de los 3 modelos 3D (Jeep, Alfa Romeo, y Fiat) en la escena, previos a la aplicación final de la iluminación global.

- La ventana flotante *Lighting* muestra la pestaña de *Environment*.
- El *Skybox Material* actual está configurado como *Default-Skybox*, generando una iluminación plana y un fondo grisáceo en el horizonte.

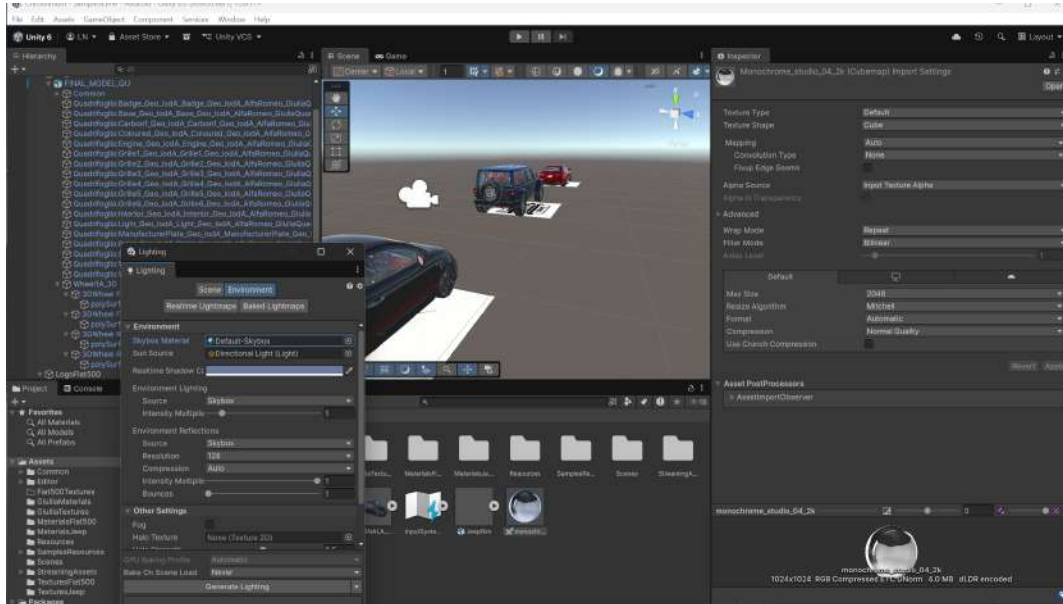


Figura 24: Escena con skybox por defecto.

7.29. Aplicación del skybox HDRI

Al reemplazar el skybox, los materiales metálicos responden con reflejos especulares más precisos, alineados con un entorno tipo showroom.

- Aplicación del material HDRI fotorealista a la iluminación global de la escena en Unity.
 - En la ventana flotante *Lighting*, se reemplaza el cielo por defecto asignando el *Skybox.Studio.Material* en el parámetro *Skybox Material*.
 - En tiempo real, la carrocería roja del Alfa Romeo reacciona al nuevo entorno, generando reflejos metálicos precisos y destellos especulares (*specular highlights*) característicos de un showroom.



Figura 25: Skybox HDRI aplicado a la escena.

7.30. Ajuste de exposición del skybox

La exposición controla el balance entre reflejos y detalle en texturas metálicas, evitando sobreexposición en superficies brillantes.

- Ajuste final del mapa ambiental (Skybox) sobre la escena 3D global.
 - En el panel del Inspector, se configura el parámetro *Exposure* (Exposición) del *Skybox.Studio.Material* a un valor de 0.81.
 - Se observa en la vista de escena cómo los 3 vehículos reaccionan a este ajuste, atenuando los brillos excesivos y mejorando el contraste general de las texturas metálicas.

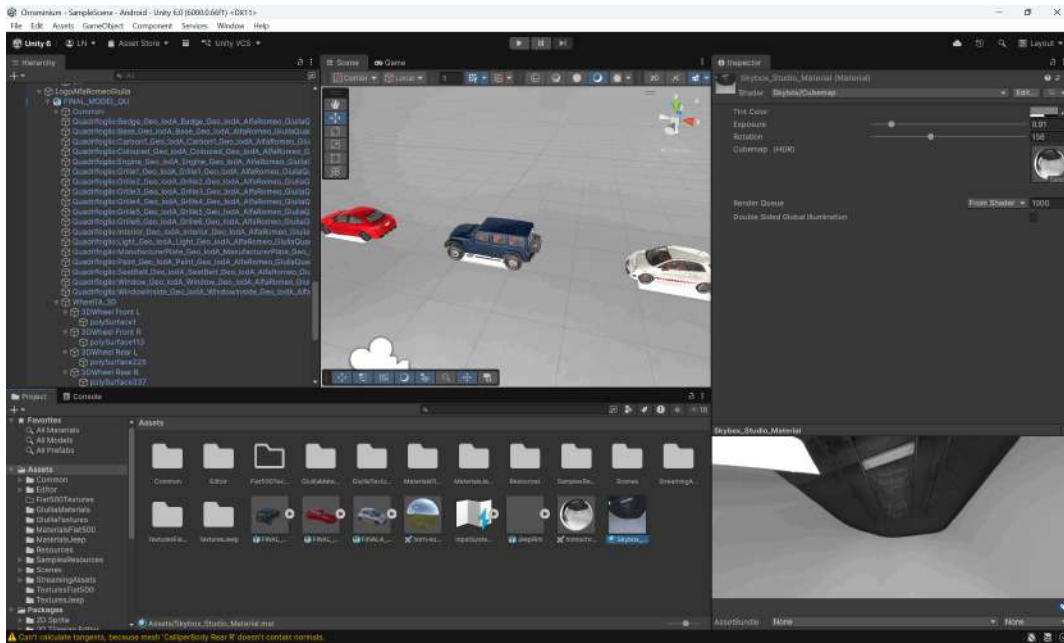


Figura 26: Ajuste de exposición del skybox para balance visual.

7.31. Asignación de materiales interiores PBR

El interior del vehículo requiere materiales mate con control de *Smoothness* para evitar brillos irreales. Esto mejora la percepción de plástico y cuero en RA.

- Asignación de texturas y materiales para el interior del modelo Fiat 500 Abarth.
 - Se observa en la Jerarquía (*Hierarchy*) la estructura interna del modelo 3D.
 - En la vista de Escena (*Scene*), se detalla el volante y el tablero con materiales tipo plástico mate.
 - En el panel del Inspector, se visualiza la configuración del shader *Standard* ajustando el mapa de Albedo y el *Smoothness* (Suavidad) del material interior.



Figura 27: Materiales interiores con control de suavidad.

7.32. Migración a URP y shaders personalizados

URP ofrece un pipeline optimizado para dispositivos móviles y habilita Shader Graph para personalizaciones de material. La migración se realiza una vez validado el contenido base.

7.33. Instalación de Universal Render Pipeline

URP se instala desde el Package Manager para asegurar compatibilidad con el editor y facilitar actualizaciones controladas.

- Instalación del sistema de renderizado Universal Render Pipeline (URP).
 - La ventana del Package Manager descarga el paquete *Universal Render Pipeline*.
 - Este motor de renderizado avanzado es necesario para crear shaders personalizados y optimizar gráficos en móviles.

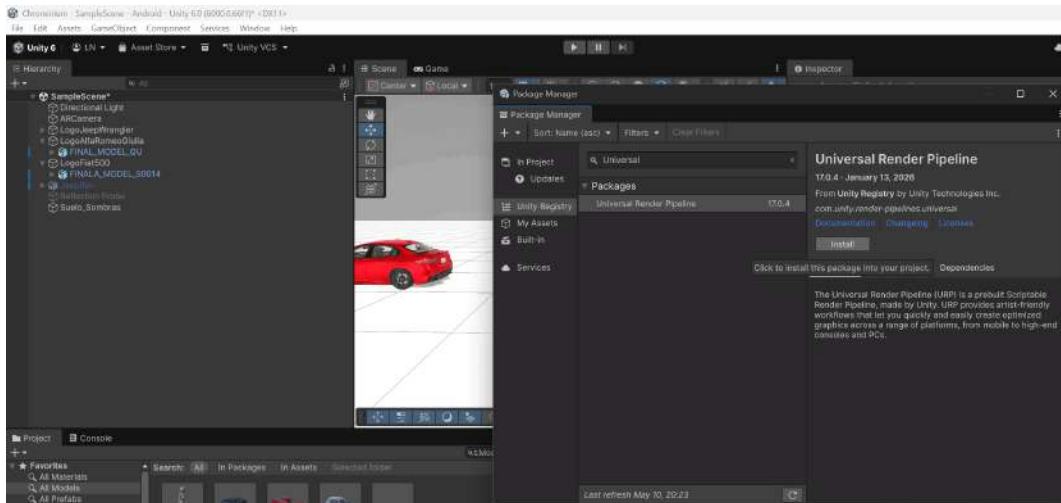


Figura 28: Instalación de URP desde Package Manager.

7.34. Creación del asset de configuración URP

El asset controla calidad, sombras y renderers. Centralizar esto en un *URP Asset* garantiza consistencia visual entre escenas.

- Generación de los archivos de configuración gráfica URP.
 - Mediante el menú contextual (*Create -> Rendering*), se crea el activo *URP Asset* (*with Universal Renderer*).
 - Este archivo administrará la calidad gráfica, las sombras y la iluminación del proyecto en AR.

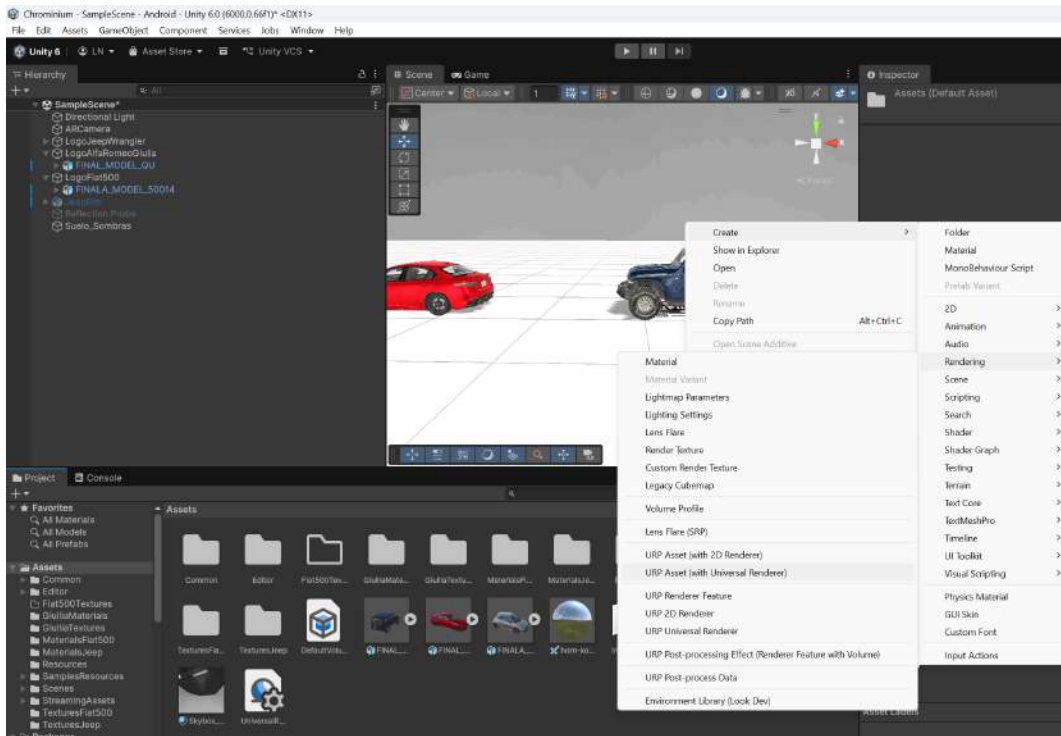


Figura 29: Creación del asset URP con Universal Renderer.

7.35. Validación de URP instalado

Se confirma la versión del paquete para evitar inconsistencias al convertir materiales y shaders.

- Verificación de la instalación del Universal Render Pipeline (URP) en el Package Manager.
 - Se observa que la versión 17.0.4 del paquete URP figura como instalada en el *Unity Registry*.
 - Este paquete es esencial para reemplazar el sistema de renderizado tradicional, permitiendo la creación del shader personalizado para atrapar sombras invisibles.

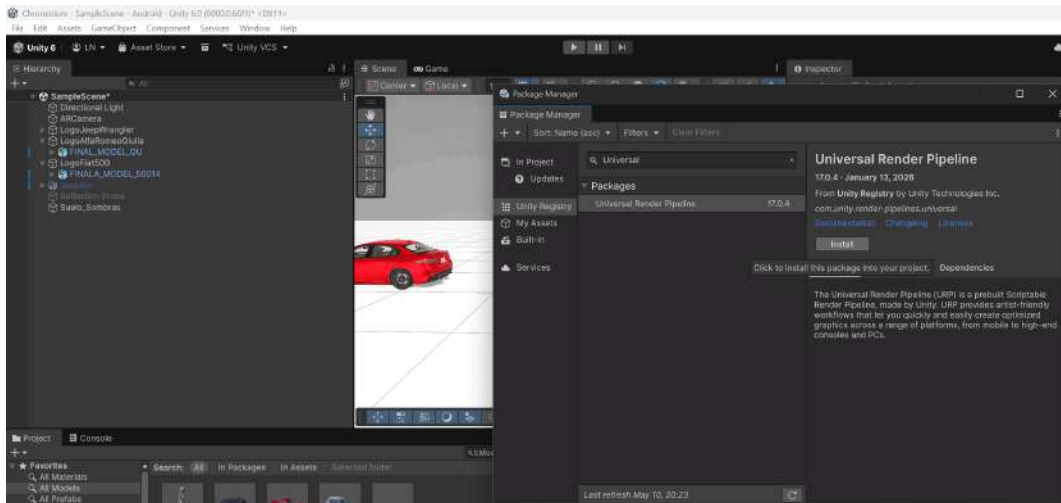


Figura 30: URP instalado y disponible en Package Manager.

7.36. Asignación de URP como pipeline global

El *Default Render Pipeline* define el pipeline activo para todo el proyecto, incluyendo escenas de RA y materiales PBR.

- Ventana de *Project Settings* asignando el motor de renderizado universal al proyecto.
 - En la sección *Graphics*, el parámetro *Default Render Pipeline* ha sido configurado con el archivo *UniversalRenderer (Universal Render Pipeline Asset)*.
 - Este cambio a nivel de arquitectura global habilita las opciones avanzadas de iluminación y sombras necesarias para el prototipo de Realidad Aumentada.

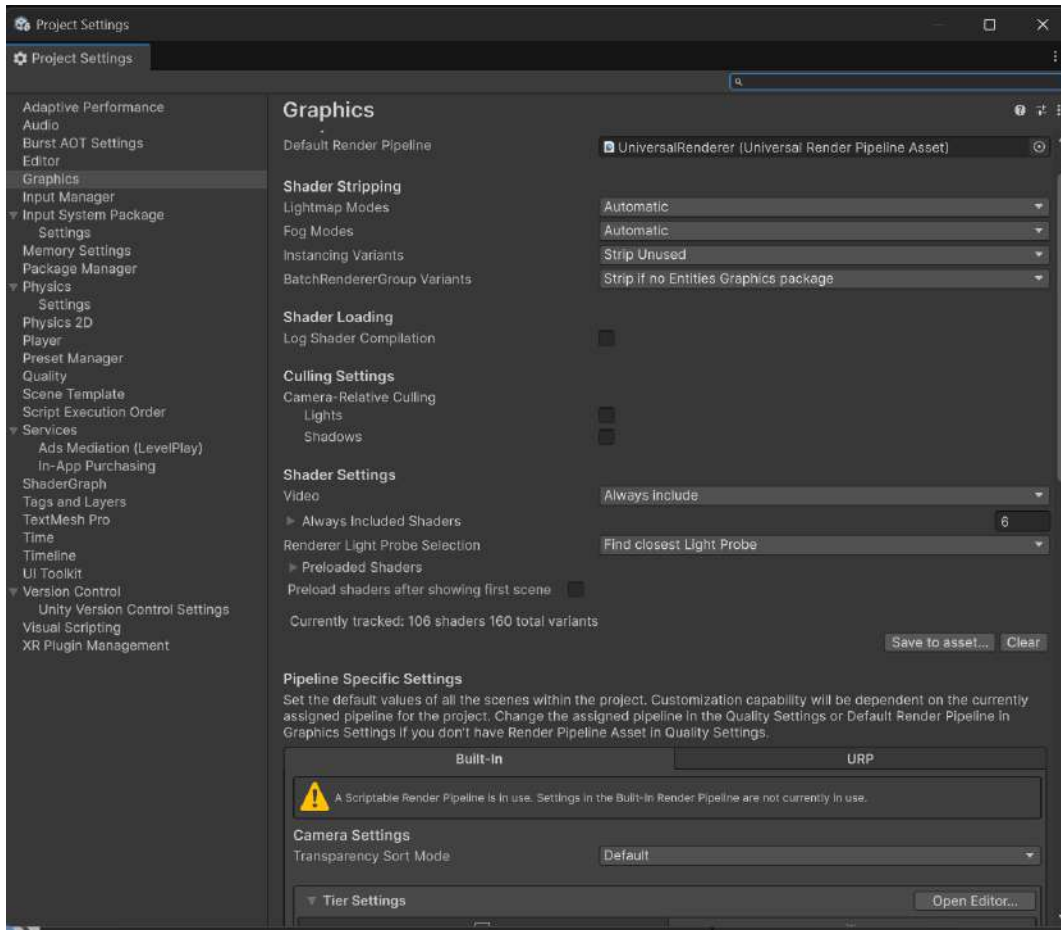


Figura 31: Asignación de URP como pipeline global.

7.37. Conversión de materiales a URP

El *Render Pipeline Converter* actualiza materiales *Standard* a equivalentes URP, evitando que los modelos queden en magenta por incompatibilidad de shaders.

- Interfaz de la herramienta *Render Pipeline Converter* de Unity.
 - Se selecciona la opción *Built-in to URP* en el menú superior para realizar la migración de los activos existentes.
 - El objetivo es actualizar de forma automática los materiales estándar y elementos gráficos creados previamente para que sean compatibles con el nuevo motor de renderizado.

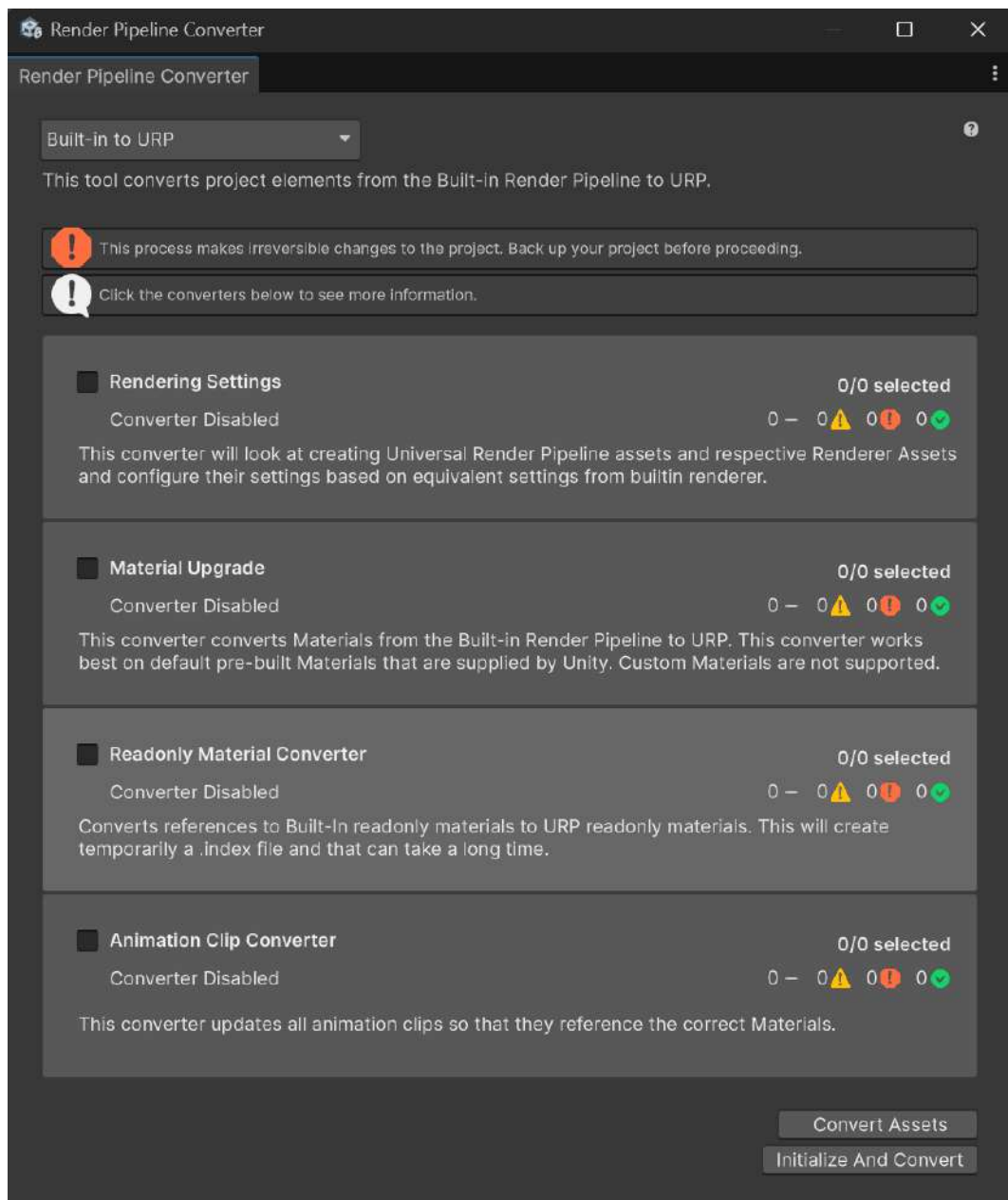


Figura 32: Configuración de conversión Built-in a URP.

7.38. Resultado de la conversión de materiales

Se revisa el listado para identificar materiales que requieren ajustes manuales, como el skybox.

- Ejecución y confirmación del conversor de materiales de URP.
 - La interfaz muestra que se seleccionaron y analizaron 80 materiales para la actualización.

- Se visualiza una lista de confirmación con marcas de verificación verdes (*ticks*) indicando que los materiales de los autos (*GiuliaMaterials*, *MaterialsJeep*) se convirtieron exitosamente.
- Se advierte un ícono de alerta en el material del Skybox, el cual requiere configuración manual posterior.

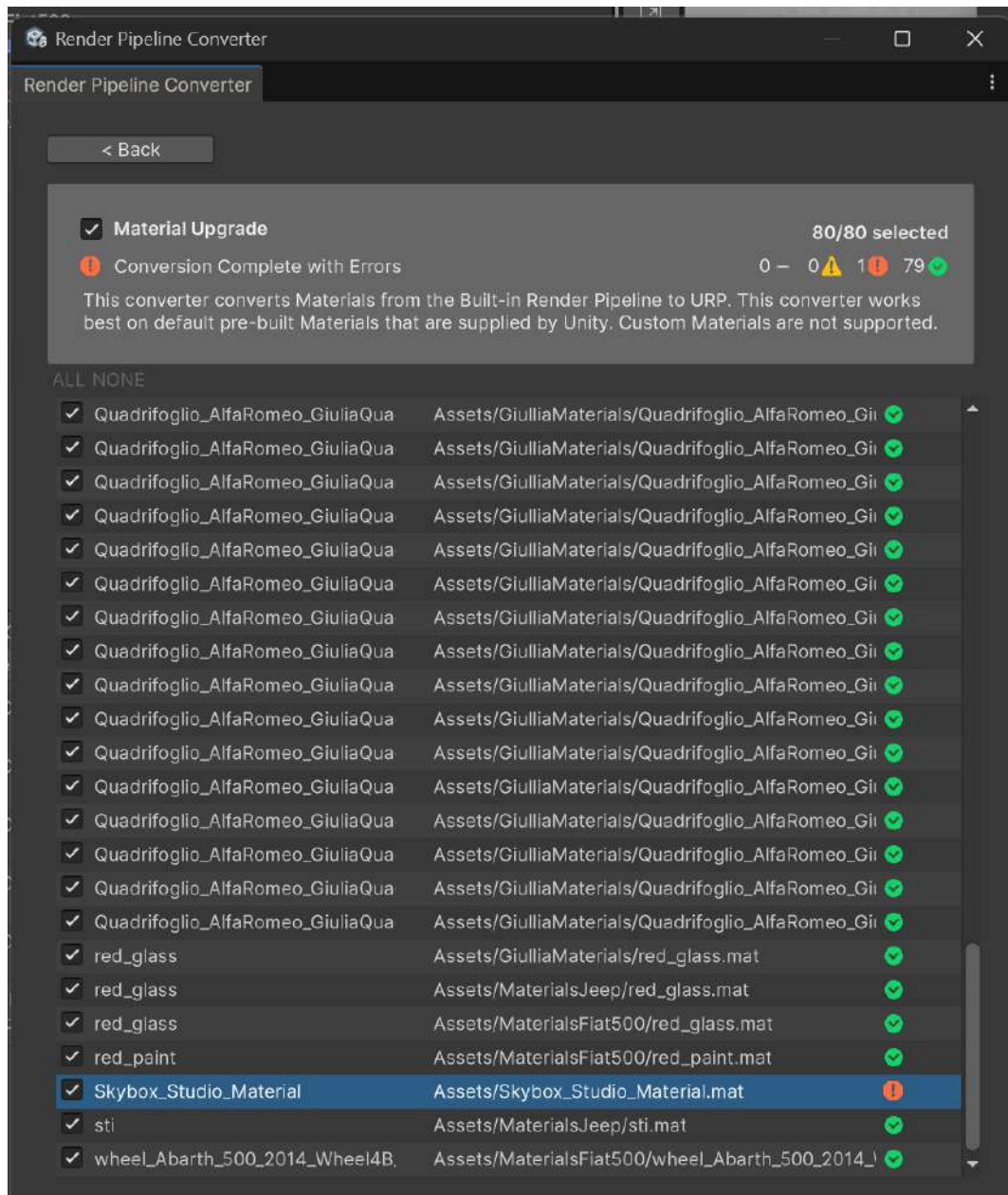


Figura 33: Resultado de conversion de materiales a URP.

7.39. Ajustes avanzados de sombras en URP

Se optimiza la calidad de sombras para mantener realismo sin sobrecargar la GPU móvil. Reducir distancia y cascadas ayuda a focalizar calidad cerca del marcador.

- Optimización avanzada del archivo de configuración gráfica del Universal Render Pipeline (*Ultra_Pipeline Asset*).
 - En la sección *Lighting*, el *Shadow Resolution* general se maximiza a 4096 píxeles.
 - En la sección *Shadows*, se reduce la *Max Distance* a 150 metros y el *Cascade Count* se fija en 1.
 - Se habilita la opción *Soft Shadows*, crucial para renderizar bordes de sombra suaves y realistas sobre los marcadores AR en dispositivos móviles sin saturar la GPU.

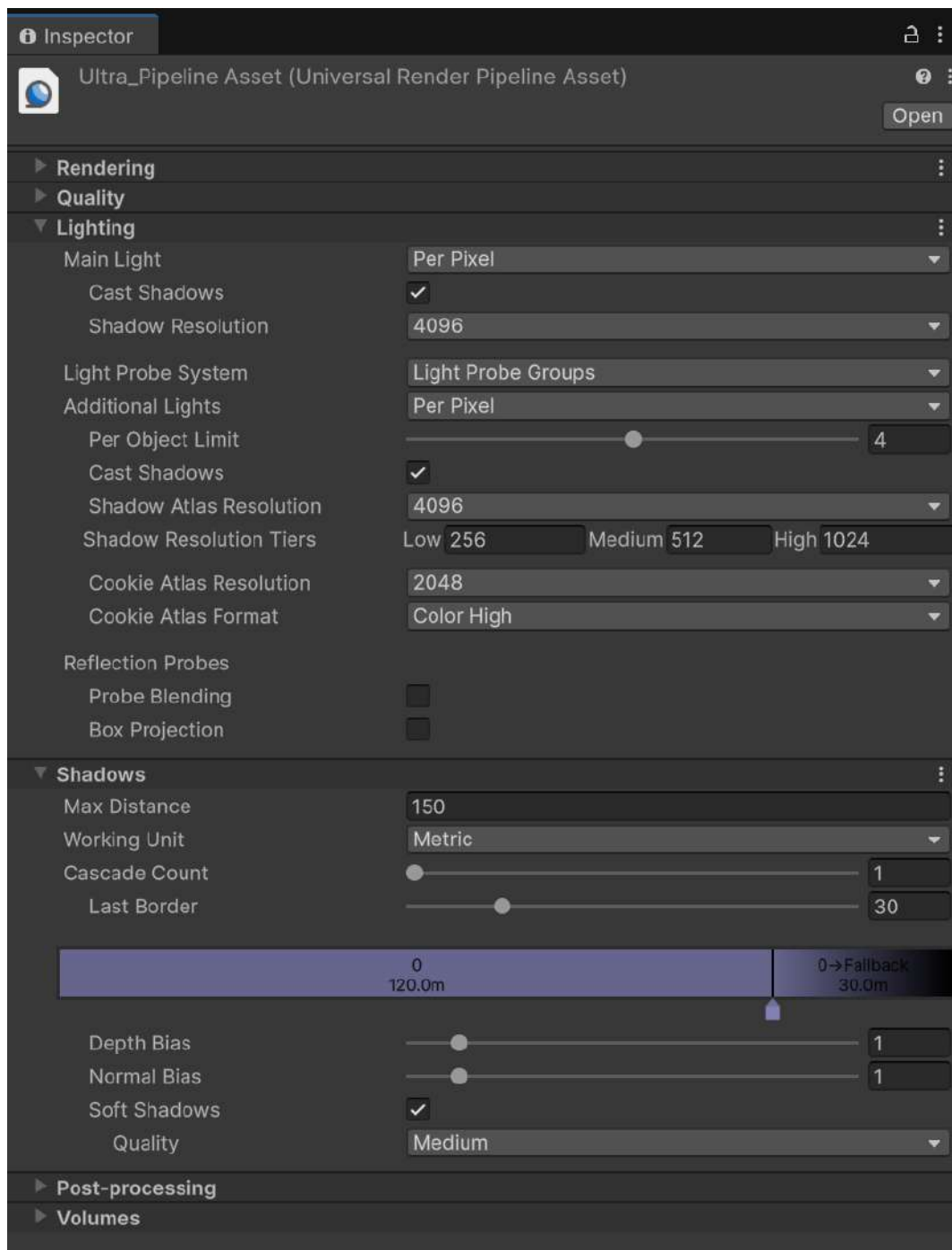


Figura 34: Ajustes de sombras en el asset URP.

7.40. Prueba de shadow catcher

El material *ShadowCatcher* permite proyectar sombras sin mostrar el plano, reforzando la ilusión de contacto en RA.

- Prueba de renderizado e iluminación direccional en el entorno AR.

- Se observa en la Jerarquía la estructura completa del *LogoJeepWrangler* conteniendo su luz focal (*Spot Light*).
- En el Inspector, el plano *Suelo_Sombras* tiene asignado el *Material_ShadowCatcher*.
- En la vista de Escena, se comprueba que el plano blanco original es ahora invisible, mostrando únicamente la sombra dinámica negra proyectada por el Jeep Wrangler.



Figura 35: Shadow catcher aplicado al plano de sombras.

7.41. Creación del shader ShadowCatcher

El shader ShadowCatcher permite hacer invisible el plano receptor mientras conserva la sombra proyectada, mejorando el realismo en RA.

- Creación del activo (*Asset*) fundamental para atrapar sombras invisibles mediante Shader Graph.
 - Se despliega el menú contextual del panel de Proyecto (Click derecho -> *Create* -> *Shader Graph* -> *URP* -> *Lit Shader Graph*).
 - Esta acción genera un lienzo de programación visual basado en nodos, esencial para manipular el canal alfa (transparencia) del motor Universal Render Pipeline.

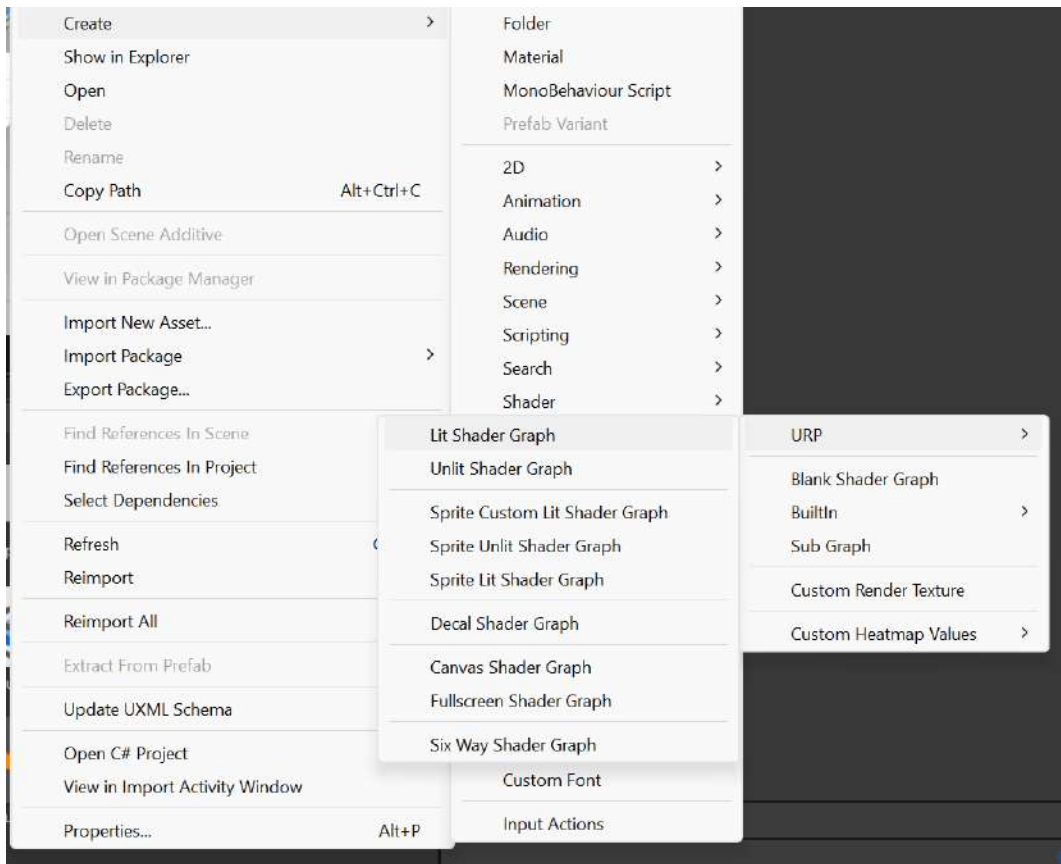


Figura 36: Creación del Shader Graph para shadow catcher.

7.42. Nodos de opacidad para shadow catcher

Se utiliza la salida de sombra como alfa para recortar el plano, dejando visible solo la sombra generada por el modelo.

- Programación visual en Shader Graph para el cálculo de opacidad (*ShadowCatcher*).
 - Se muestra el entorno de nodos del archivo *ShadowCatcher.shadergraph*.
 - Un nodo de posición global (*Position - World*) se conecta a una función personalizada (*ObtenerSombra*).
 - La salida matemática se invierte mediante un nodo *One Minus*, conectándose finalmente al canal *Alpha* del bloque maestro *Fragment*, logrando que la sombra defina la opacidad del material.

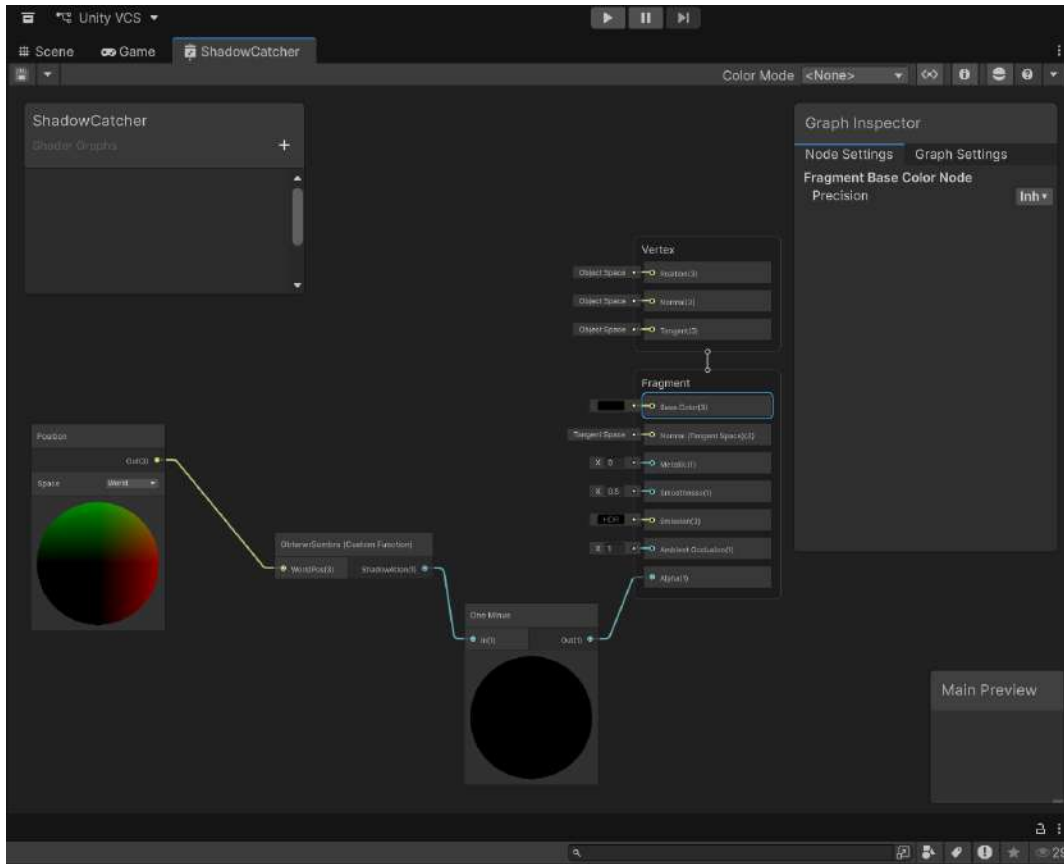


Figura 37: Shader Graph configurado para opacidad por sombra.

7.43. Validación de escena final

Se verifica la convivencia de UI, modelos, iluminación y shadow catcher en tiempo real.

- Visualización de la escena final con todos los sistemas operando simultáneamente.
 - Los modelos 3D y las interfaces de usuario (Fichas Técnicas, Sliders, Botones) conviven en el espacio AR.
 - La iluminación basada en imagen (HDRI) genera reflejos especulares sobre los vehículos.
 - El material *ShadowCatcher*, recién programado, opera exitosamente bajo el Jeep Wrangler, recortando el suelo virtual y dejando únicamente la sombra proyectada.

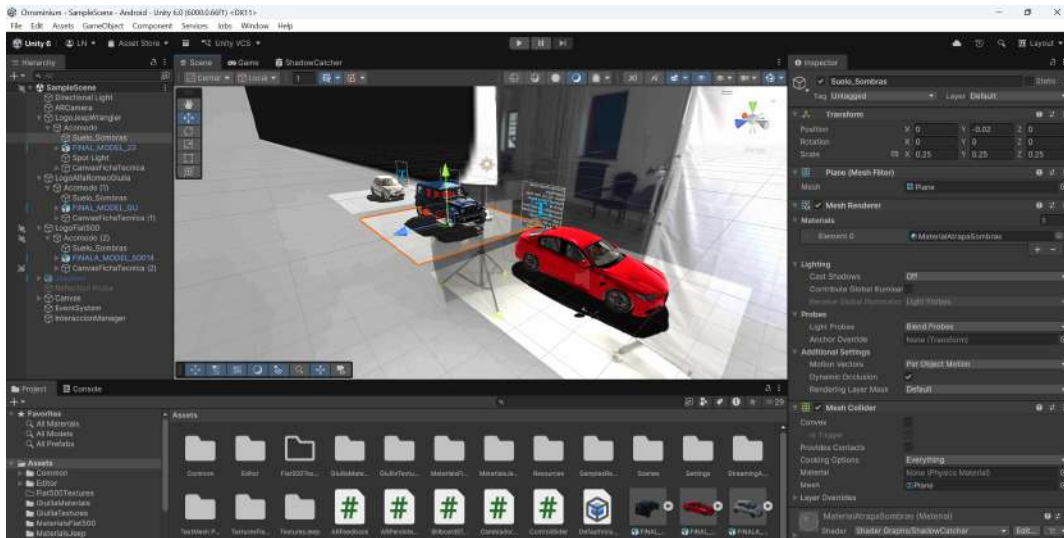


Figura 38: Escena final con UI, HDRI y shadow catcher.

7.44. Interfaz de usuario (UI)

La interfaz de usuario se mantiene minimalista para no competir con la visualización del vehículo. Los controles priorizan la escala y la navegación entre modelos mediante gestos simples.

7.45. Creación del control Slider

El *Slider* permite ajustar el tamaño del modelo con una sola mano en pantalla táctil, facilitando demostraciones en entornos de venta.

- Creación de elementos de Interfaz de Usuario (UI).
 - En el panel de Jerarquía, se navega por el menú contextual de UI para crear un *Slider*.
 - Este control deslizante funcionará como el elemento para escalar los modelos 3D en la pantalla táctil.

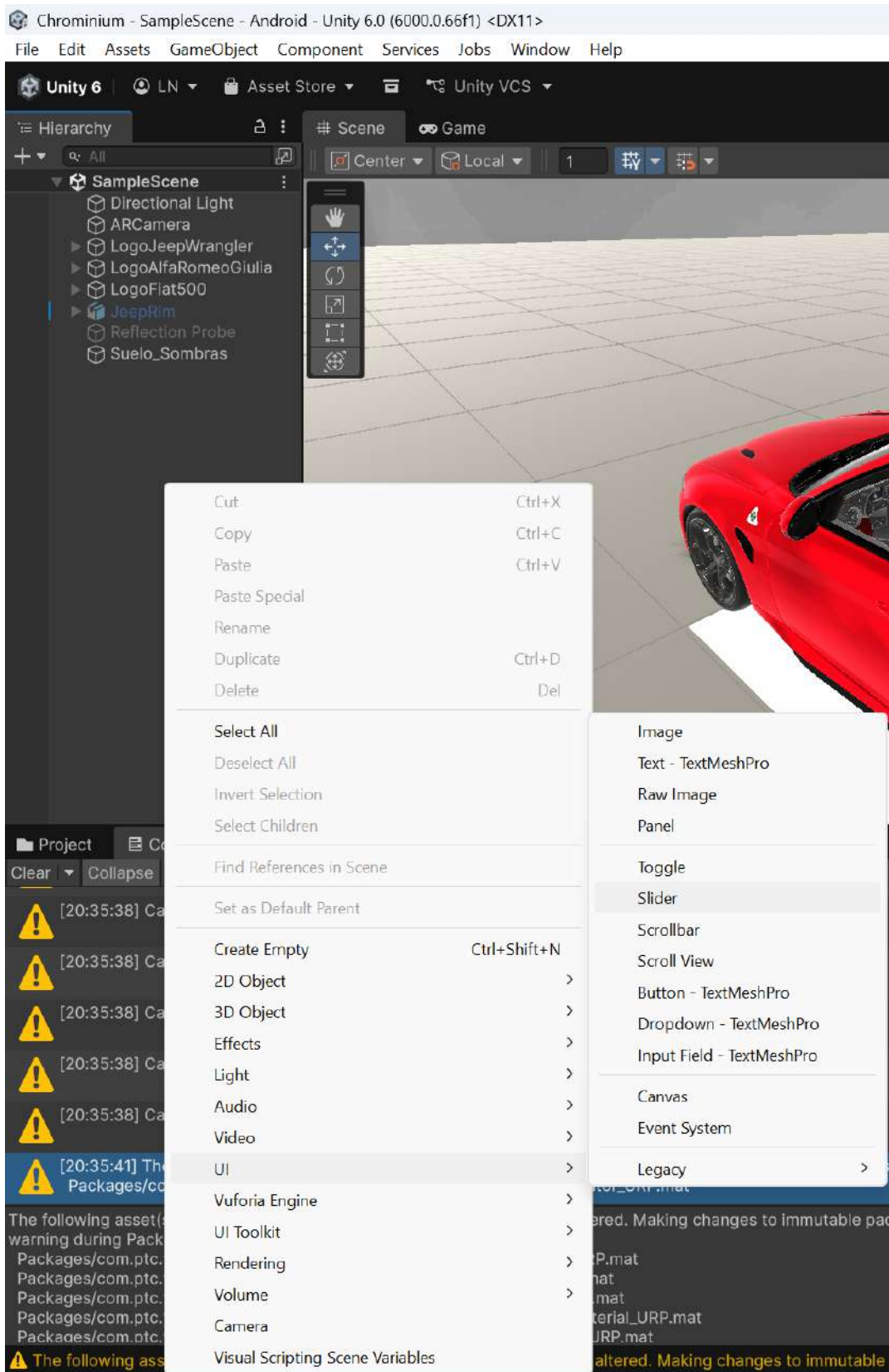


Figura 39: Creación del Slider para escalado.

7.46. Configuración de Canvas y rangos de escala

El Canvas se ajusta en modo *Screen Space* para asegurar consistencia visual en dispositivos con diferentes resoluciones.

- Creación del elemento Canvas y Slider para la Interfaz de Usuario (UI).
 - En la Jerarquía, se estructura el Canvas principal conteniendo el objeto *SliderTamaño*.
 - En la vista de Escena 2D, se manipula el anclaje central del control deslizante.
 - El Inspector detalla la configuración del componente Slider, estableciendo los valores *Min Value* (0.2) y *Max Value* (2) para limitar el rango de escalado de los modelos.

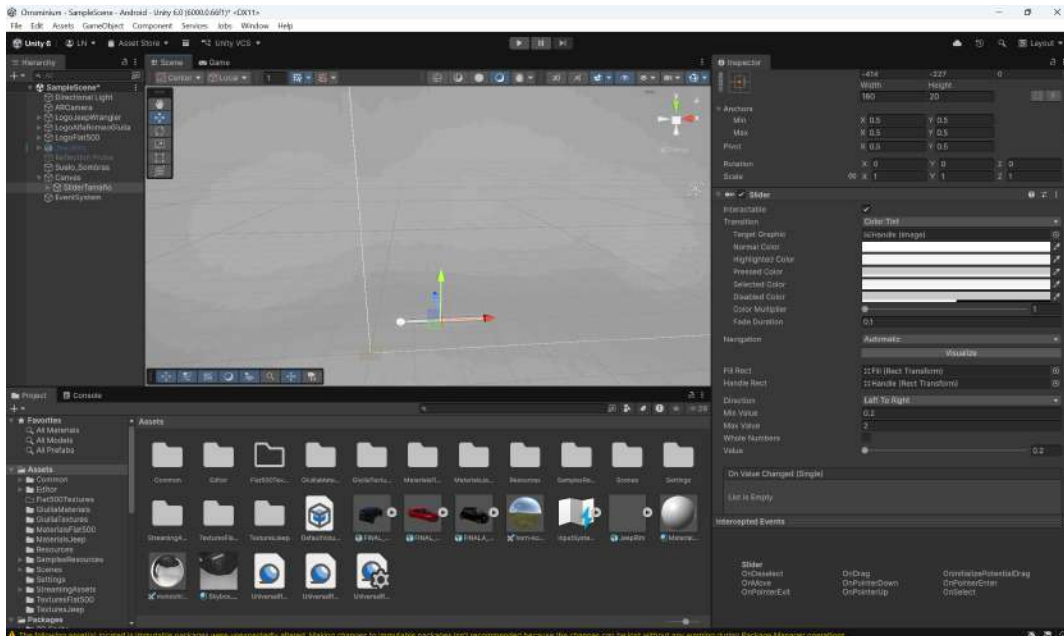


Figura 40: Canvas con Slider y rangos de escala.

7.47. Validación visual en URP y eventos de Vuforia

Se comprueba que los modelos rendericen correctamente en URP y que los eventos de tracking estén enlazados a scripts de persistencia.

- Visualización de los modelos actualizados tras la conversión a URP y configuración de scripts de interacción.
 - Los vehículos ahora lucen los shaders PBR (Physically Based Rendering) propios del motor universal.
 - En el Inspector del marcador *FINAL_MODEL_50014* (Fiat 500), se visualizan los componentes *Default Observer Event Handler* para gestionar el rastreo de Vuforia.

- [illegible]

7.48. Jerarquías de acomodo para escalado uniforme

- Estructuración de jerarquías para el control de transformaciones y escalado de los modelos AR.

- 49

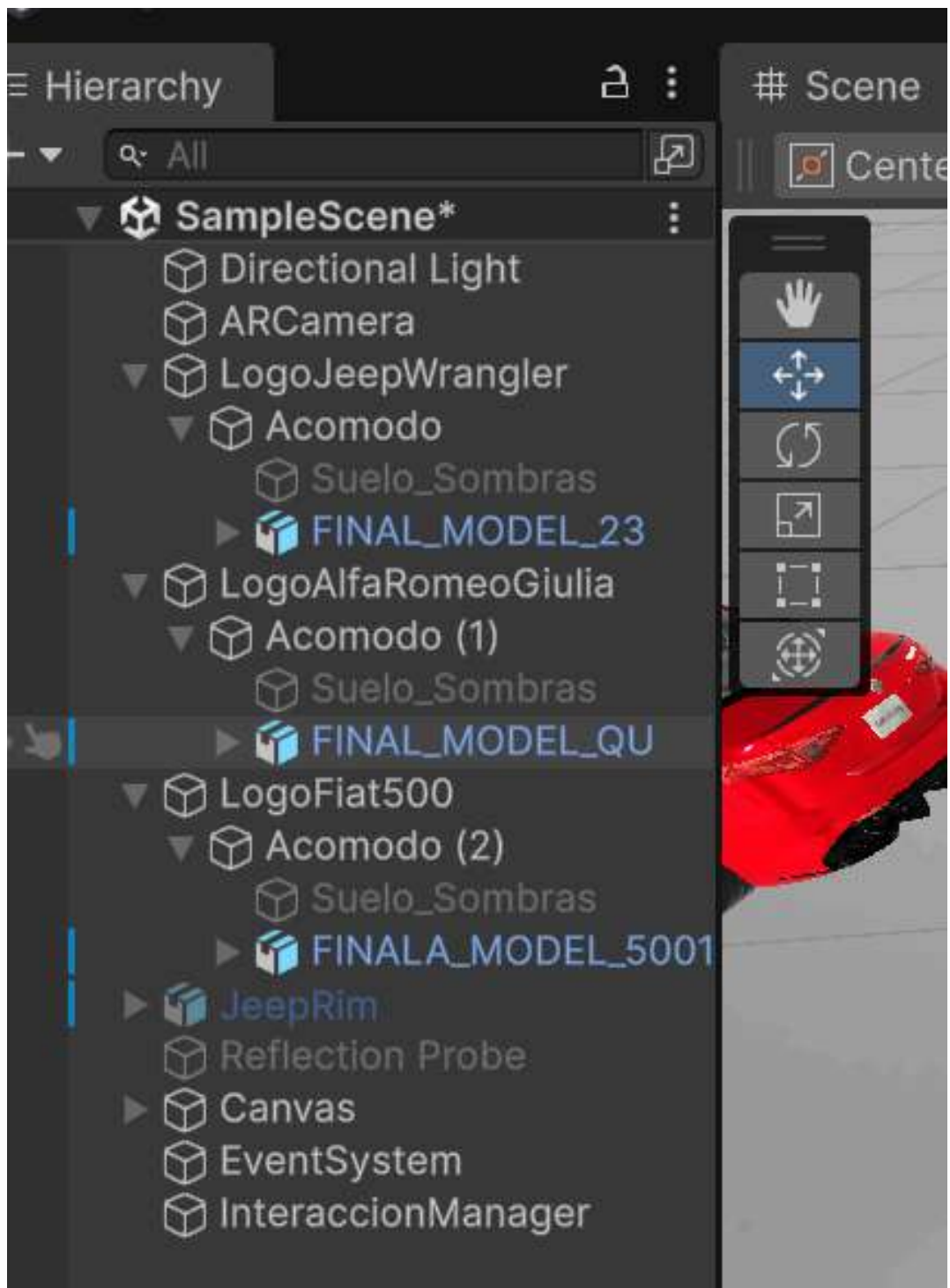


Figura 42: Jerarquias de acomodo para transformaciones.

7.49. Configuración de scripts de escala táctil

Se definen rangos mínimos y máximos para evitar distorsión y mantener consistencia visual en dispositivos móviles.

- Configuración de parámetros dentro de los scripts personalizados de Interacción UI.
 - Se observa el panel Inspector detallando dos scripts: *Control Slider* y *Pinch Zoom*.
 - En *Modelos A_Escalar*, se asignan las raíces de los 3 modelos 3D.
 - En *Objetos A_Escalar* (dentro de Pinch Zoom), se vinculan los 3 nodos *Acomodo* creados previamente.
 - Se establecen los rangos de tolerancia táctil: *Escala Minima* (0.1) y *Max Escala* (4 o 10 dependiendo del script) para evitar distorsiones del modelo en el dispositivo móvil.

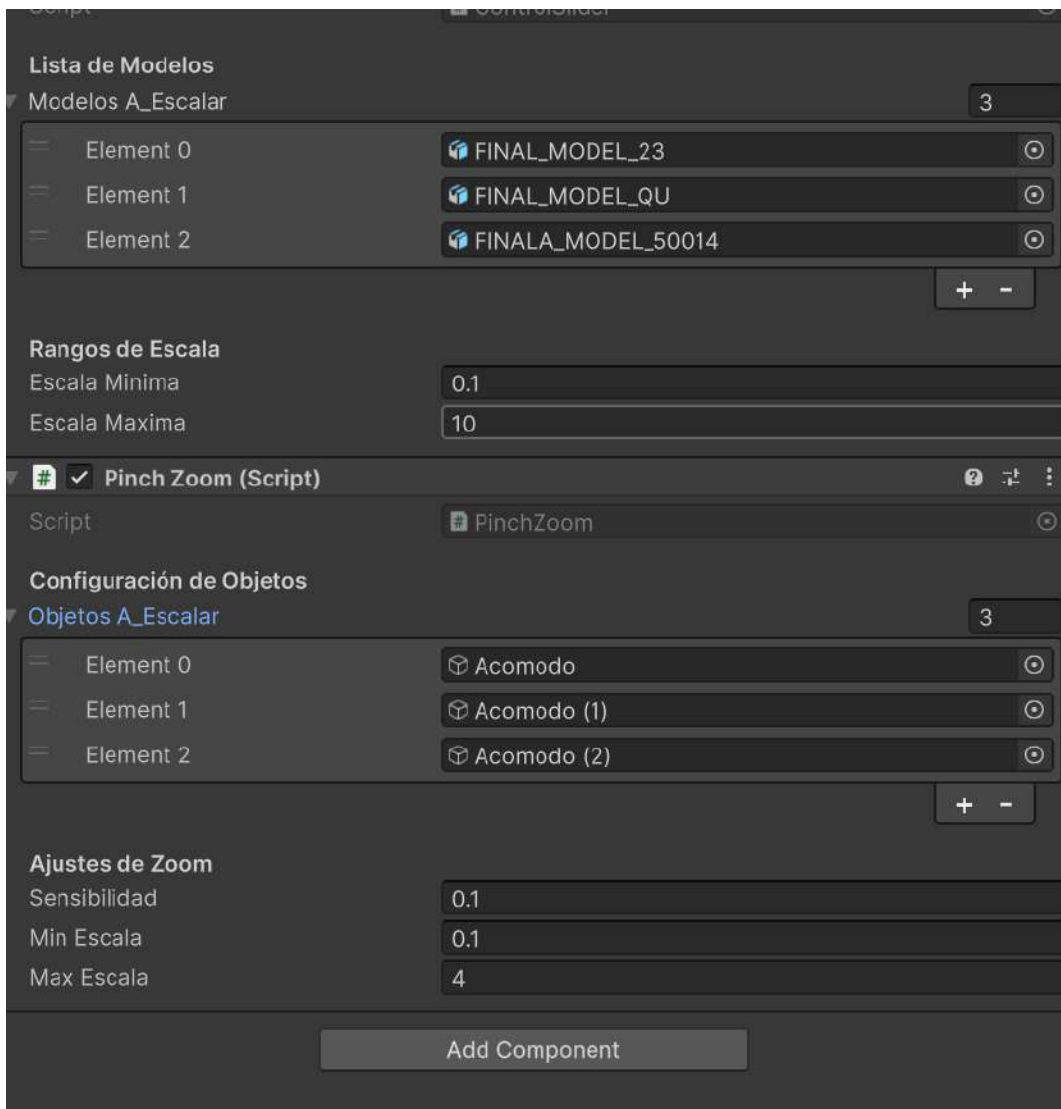


Figura 43: Scripts de escala y pinch-zoom configurados.

7.50. Reescalado global de nodos Acomodo

Se normaliza la escala de los nodos padre para que todos los modelos respondan de forma consistente al Slider y a los gestos táctiles.

- Verificación de los parámetros de transformación globales (Reescalado).
 - En la vista de Escena, los 3 vehículos se muestran escalados a un tamaño relativo adecuado para la visualización AR.
 - En el Inspector, se observa el objeto padre *Acomodo* seleccionado.
 - El componente *Transform* revela una escala (*Scale*) uniforme de X:3, Y:3, Z:3, asegurando que tanto el modelo 3D como su plano atrapasombras mantengan la misma proporción geométrica.

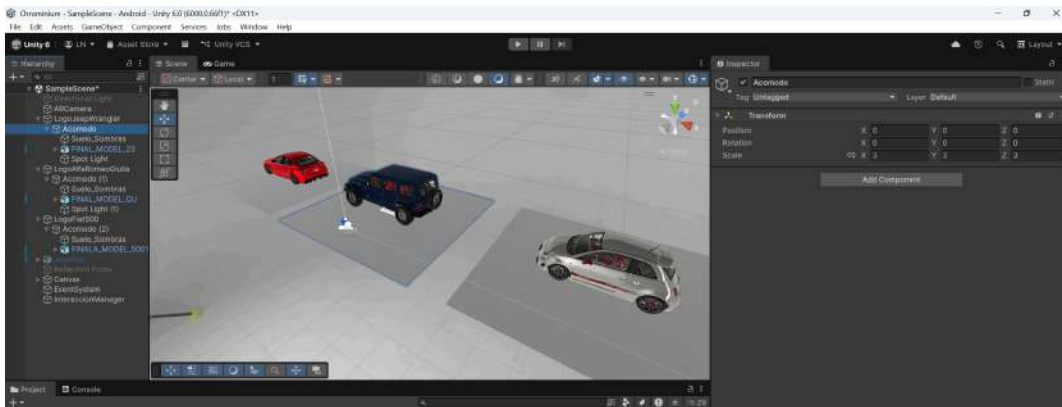


Figura 44: Reescalado uniforme de nodos Acomodo.

7.51. Materiales base para cambio de color

Se identifican los materiales de carrocería para permitir ajustes dinámicos sin afectar otros materiales secundarios.

- Exploración y selección del material base de la carrocería del Jeep Wrangler.
 - La carpeta *MaterialsJeep* se despliega en el panel inferior mostrando los materiales PBR generados.
 - El material *_J_MI_1355060001_004* es seleccionado.
 - En el Inspector, se confirman los parámetros del shader *Universal Render Pipeline/Lit*, con un *Base Map* color azul oscuro y un nivel alto de *Smoothness* para simular pintura automotriz metálica.



Figura 45: Material base de carrocería del Jeep.

7.52. Material base del Fiat 500 Abarth

Se selecciona el material principal para habilitar cambios de color en tiempo real desde la UI.

- Exploración y selección del material base de la carrocería del Fiat 500 Abarth.
 - La carpeta *MaterialsFlat500* está activa en el explorador de Assets.
 - El material *_Abarth_500_2014Base_Material1* está seleccionado.
 - El Inspector muestra el canal de *Albedo* (Base Map) en color gris/plata claro, listo para ser manipulado dinámicamente mediante el script de cambio de color.



Figura 46: Material base del Fiat 500 Abarth.

7.53. Material base del Alfa Romeo Giulia

Se valida el mapa metalico para controlar reflejos y barniz en la pintura de alto brillo.

- Exploración y selección del material base de la carrocería del Alfa Romeo Giulia.
 - La carpeta *GiuliaMaterials* está activa mostrando sus respectivos *Assets*.
 - El material *Quadrifoglio_AlfaRomeo_GiuliaQuadrifoglio_2017Paint_Material1* está seleccionado.
 - El Inspector muestra el color base ajustado a un rojo brillante, con el *Metallic Map* configurado en un valor de 0.621 para definir la reflectividad del barniz.



Figura 47: Material base del Alfa Romeo Giulia.

7.54. Configuración del script de cambio de color

El script agrupa mallas por vehículo para aplicar un color uniforme sin afectar detalles como cristales o interiores.

- Configuración del Script *CambiadorColor* para la interacción de la Interfaz de Usuario.
 - En el Inspector del script, se vinculan los 3 nodos *Acomodo* en la sección de Referencias de Activación.
 - La sección *Mallas de las Carrocerías* muestra arreglos (*Arrays*) detallados.
 - En *Carroceria Jeep*, se listan 11 mallas independientes (*Mesh Renderers*) correspondientes a las puertas, cofre y cajuela, permitiendo que el script altere el color de todas las piezas simultáneamente.

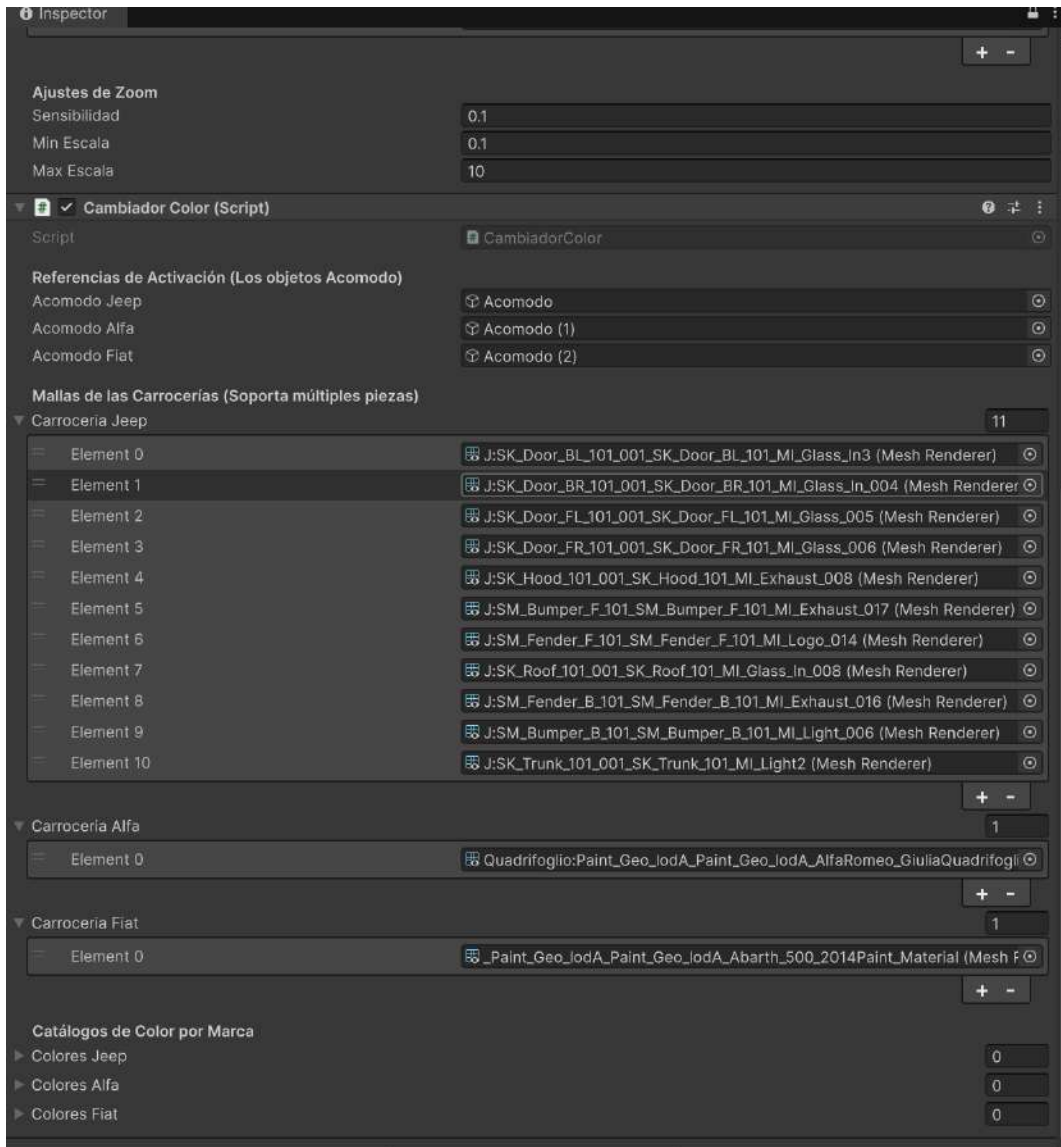


Figura 48: Script de cambio de color con mallas asignadas.

7.55. Botón de cambio de color y nuevo escaneo

Se integra el botón con el evento de UI para ciclar colores sin tocar el modelo directamente.

- Integración y configuración del botón de la interfaz gráfica *Nuevo Escaneo / Cambio de Color*.
 - En la Jerarquía, dentro del elemento *Canvas*, se localiza el *BotonColor*.
 - En la vista de Escena, el botón flotante amarillo es posicionado sobre los elementos 3D.
 - En el Inspector, el componente *Button* (Botón) define un evento en *On Click ()*, ejecutando la función *CambiatorColor.AlternarColorAuto* del *InteraccionManager* para ciclar los colores del modelo activo.

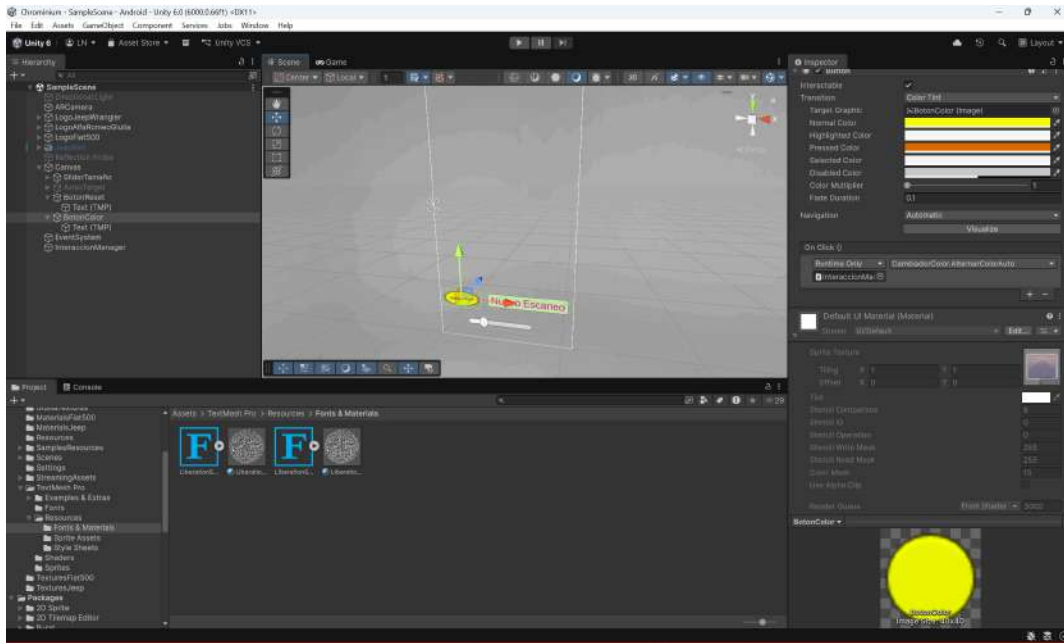


Figura 49: Boton de cambio de color con evento asociado.

7.56. Edición de texto con TextMeshPro

Las fichas técnicas se renderizan en *World Space* para mantener legibilidad junto al modelo, incluso al moverse alrededor del objetivo.

- Edición de texto tridimensional usando TextMeshPro para la Ficha Técnica.
 - En la Jerarquía, se visualiza el objeto *TextoSpecs* anidado dentro del Canvas de la Ficha Técnica.
 - En la ventana del Inspector, se redactan los datos técnicos del vehículo en el campo *Text Input*.
 - Se observa el recuadro delimitador amarillo (*Bounding Box*) en la vista de escena ajustando el texto junto al vehículo.

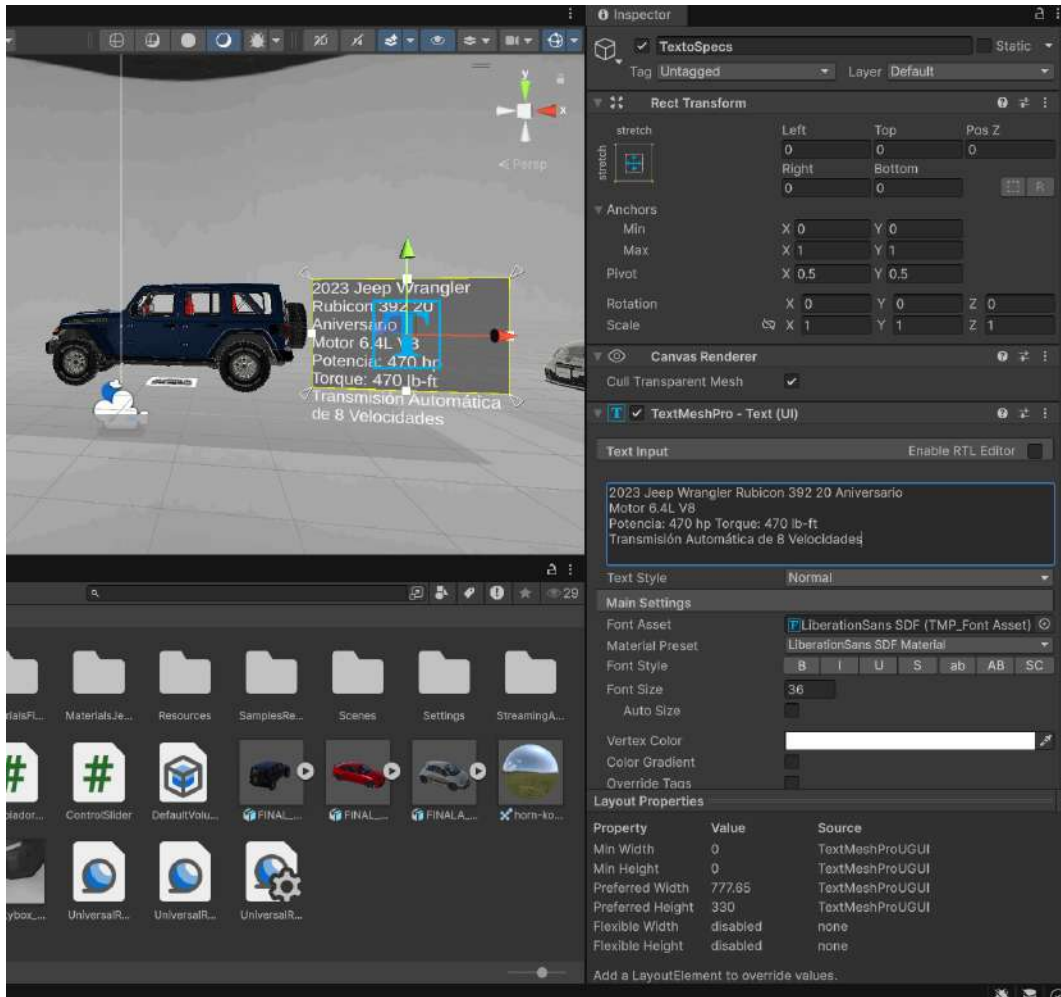


Figura 50: Edición de fichas técnicas con TextMeshPro.

7.57. Fichas técnicas en World Space

Cada vehículo tiene su propio canvas para mantener la ficha vinculada al modelo correcto durante el tracking.

- Previsualización general de los lienzos de información (Fichas Técnicas) de los tres modelos.
 - La Jerarquía muestra que cada vehículo cuenta con su propio *CanvasFichaTecnica* subordinado a su respectivo *Acomodo*.
 - En la vista de Escena, se observan los 3 paneles de texto translúcidos flotando al lado del Jeep, Alfa Romeo y Fiat.
 - Estos elementos UI operan en el espacio del mundo tridimensional (*World Space*), permitiendo al usuario caminar alrededor del modelo y leer sus especificaciones.

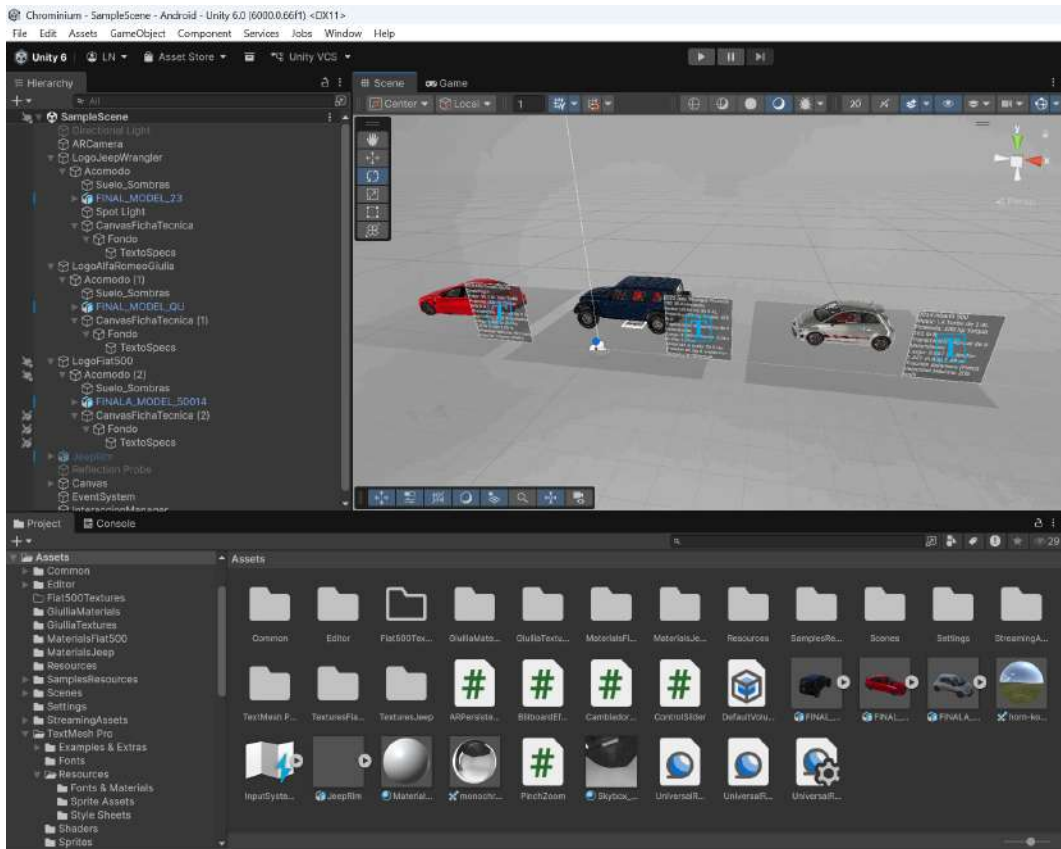


Figura 51: Fichas técnicas ancladas a cada modelo.

7.58. Botón de ficha técnica

Se integra un botón dedicado para mostrar u ocultar especificaciones bajo demanda, manteniendo la escena limpia.

- Configuración del botón de Interfaz de Usuario (UI) para la visualización de Fichas Técnicas.
 - En la Jerarquía, se selecciona el objeto *BotonInfo* (nombrado visualmente como Ficha Técnica).
 - En el panel del Inspector, se configura el componente *Button*.
 - El evento *On Click ()* está enlazado a la función *CambiadorColor.AlternarVisibilidadFichas*, permitiendo al usuario del celular mostrar u ocultar el panel de especificaciones del auto escaneado.



Figura 52: Botón para mostrar u ocultar ficha técnica.

7.59. Catálogos de color por vehículo

Se definen paletas de color con nombres comerciales para reproducir acabados reales y permitir selección precisa.

- Configuración de catálogos de colores personalizados en el script *InteraccionManager*.
 - Se observa el panel del Inspector detallando las listas de colores (*Colores Jeep*, *Colores Alfa*, *Colores Fiat*).
 - Cada elemento del arreglo define un *Nombre Comercial* y un selector *Color Hex* que establece el tono RGB exacto de fábrica.
 - Estos colores se aplicarán dinámicamente a las mallas de los vehículos durante la ejecución.

▼ Colores Jeep 6

▼ Hydro Blue

Nombre Comercial Hydro Blue

Color Hex

▼ High Velocity

Nombre Comercial High Velocity

Color Hex

▼ Sarge

Nombre Comercial Sarge

Color Hex

▼ Firecracker Red

Nombre Comercial Firecracker Red

Color Hex

▼ Bright White

Nombre Comercial Bright White

Color Hex

▼ Silver Zynith

Nombre Comercial Silver Zynith

Color Hex

+ -

▼ Colores Alfa 4

▼ Rosso Competizione

Nombre Comercial Rosso Competizione

Color Hex

▼ Blu Montecarlo

Nombre Comercial Blu Montecarlo

Color Hex

▼ Grigio Silverstone

Nombre Comercial Grigio Silverstone

Color Hex

▼ Nero Vulcano

Nombre Comercial Nero Vulcano

Color Hex

+ -

▼ Colores Fiat 3

▼ Grigio Campovolo

Nombre Comercial Grigio Campovolo

Color Hex

7.60. Submenú de selección de color

El submenú permite elegir un tono específico en lugar de un ciclo aleatorio, mejorando control y experiencia del usuario.

- Diseño y estructuración del submenú interactivo para la selección visual de colores.
 - En la Jerarquía, se muestra el objeto *PanelColores* desplegado, conteniendo múltiples elementos hijos denominados *Muestra* (*Muestra0* a *Muestra6*).
 - En la vista de Escena, se observa la alineación horizontal automática de los botones circulares blancos generados por el componente *Horizontal Layout Group*.
 - El Inspector confirma la configuración del *Layout* con un espaciado (*Spacing*) de 15 y alineación *Middle Center*.



Figura 54: Submenú de selección de color con layout horizontal.

7.61. Previsualización de la nueva interfaz

Se revisa la integración del submenú y se detectan referencias faltantes para corregir eventos de UI.

- Previsualización de la nueva interfaz de usuario ensamblada en la escena principal.
 - El *PanelColores* se integra bajo el botón principal de *Cambiar Color*.
 - El botón *Cambiar Color* ha sido reconfigurado; el evento *On Click ()* muestra una advertencia de referencia faltante (*Missing CambiadorColor.AlternarColorAuto*) que requerirá ser reasignada al nuevo sistema de submenú.
 - Esta disposición permite al usuario seleccionar tonos específicos en lugar de ciclar aleatoriamente.

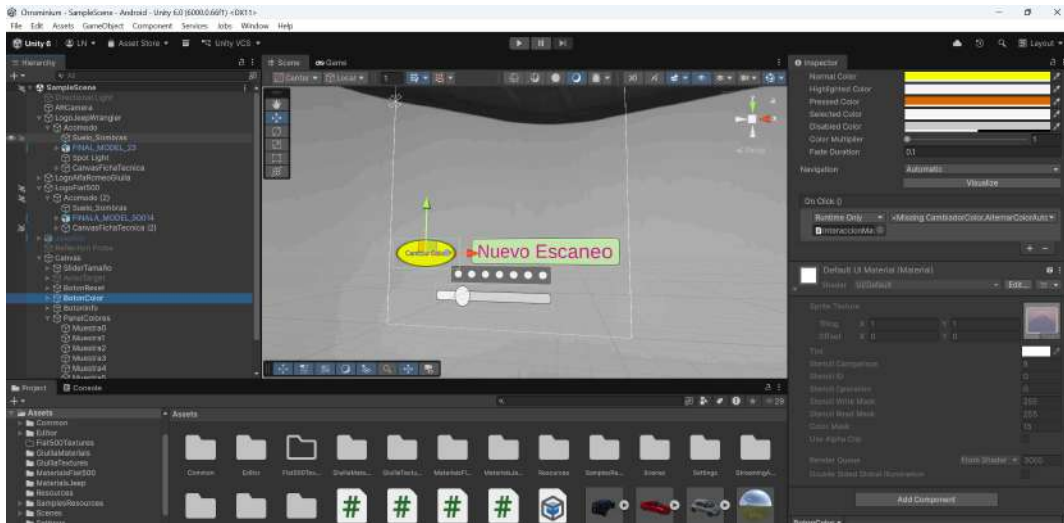


Figura 55: Interfaz con submenu de colores integrado.

7.62. Pantalla de onboarding

El onboarding guía al usuario antes de activar tracking, evitando que la cámara busque targets sin contexto.

- Creación y configuración de la pantalla inicial de bienvenida e instrucciones (*Onboarding*).
 - Un panel negro translúcido ocupa la pantalla completa de la Escena, superponiéndose a la cámara AR.
 - Se utilizan elementos TextMeshPro para el título *CHROMINIUM* y las instrucciones de uso numeradas.
 - En el Inspector, el botón *INICIAR SHOWROOM* está enlazado a la función *OnboardingManager.IniciarShowroom*, encargada de ocultar esta interfaz y activar el rastreo de marcadores.



Figura 56: Onboarding inicial con botón de inicio.

7.63. Feedback de audio en detección

El sonido de confirmación refuerza la respuesta de tracking para el usuario en un entorno con ruido visual.

- Asignación de retroalimentación auditiva (*Feedback*) mediante componentes de audio.
 - En el panel de *Assets*, se selecciona el archivo de audio importado *SuccessSoundEffect*.
 - El Inspector muestra el componente *Audio Source* configurado con este clip.
 - El script *AR Feedback* se añade para disparar este sonido de éxito cada vez que la cámara detecta y empareja correctamente uno de los logotipos.



Figura 57: Feedback de audio al detectar un target.

7.64. Color por defecto y validación final

Se define un color inicial para asegurar consistencia visual al iniciar el tracking y evitar materiales sin asignación.

- Validación final del *InteraccionManager* y adición de colores por defecto (*Default Colors*).
 - Se expande la lista *Colores Fiat* en el Inspector para añadir un cuarto elemento.
 - Se configura el color *Blanco Estelar* como tono base de arranque, asignando su Nombre Comercial y código Hexadecimal.
 - Se verifica que las Referencias de Activación estén correctamente enlazadas a los objetos del Canvas.

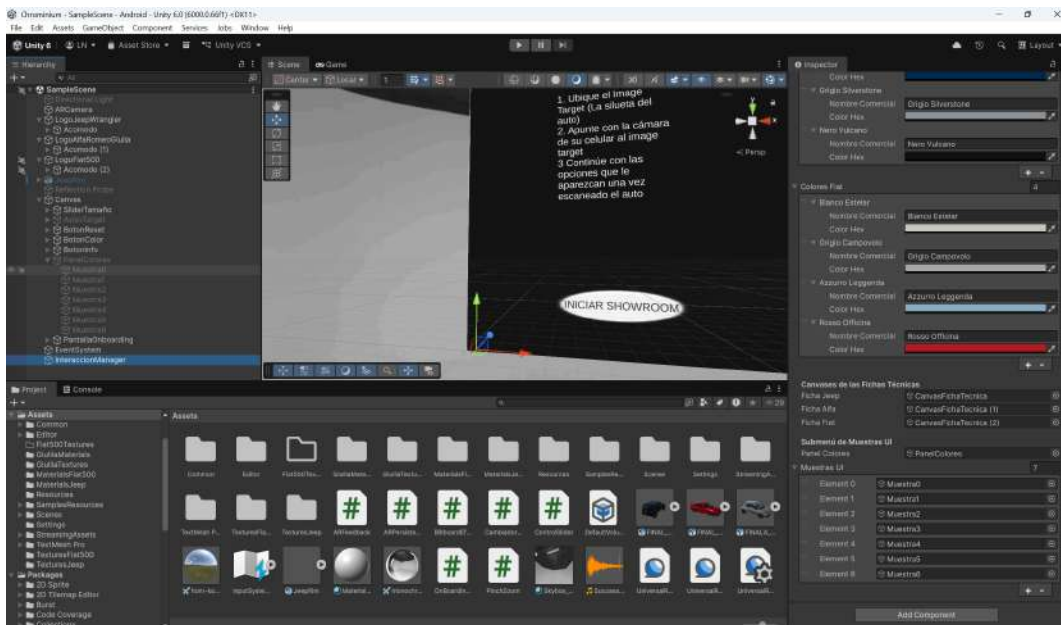
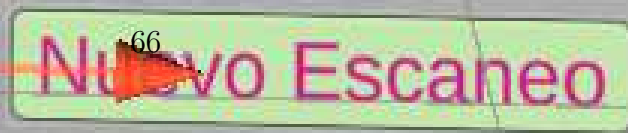


Figura 58: Color por defecto configurado en *InteraccionManager*.

7.65. Ajuste final de UI en modo portrait

Se reubican controles para ergonomía móvil y evitar que el contenido tape el vehículo.

- Rediseño y anclaje final de los elementos de la interfaz de usuario en modo *Portrait* (Vertical).
 - La vista de Escena (*Game View*) simula la pantalla vertical de un dispositivo móvil.
 - El *SliderTamaño* se ancla en la parte inferior central.
 - Los botones de acción principal (*Cambiar Color*, *Nuevo Escaneo*) se agrupan en el tercio inferior.
 - El icono de la *Ficha Técnica* se fija en la esquina superior derecha para no obstruir la visualización del vehículo en Realidad Aumentada.



7.66. Verificación de la AR Camera en la jerarquía

Se confirma que la escena utiliza la cámara de Vuforia antes de seguir con UI, evitando inconsistencias en el punto de vista de RA.

- Verificación de la creación de la AR Camera.
 - Se muestra nuevamente la ruta en el menú contextual (*Vuforia Engine -> AR Camera*).
 - Confirma la integración del punto de vista de la Realidad Aumentada en la estructura del proyecto.

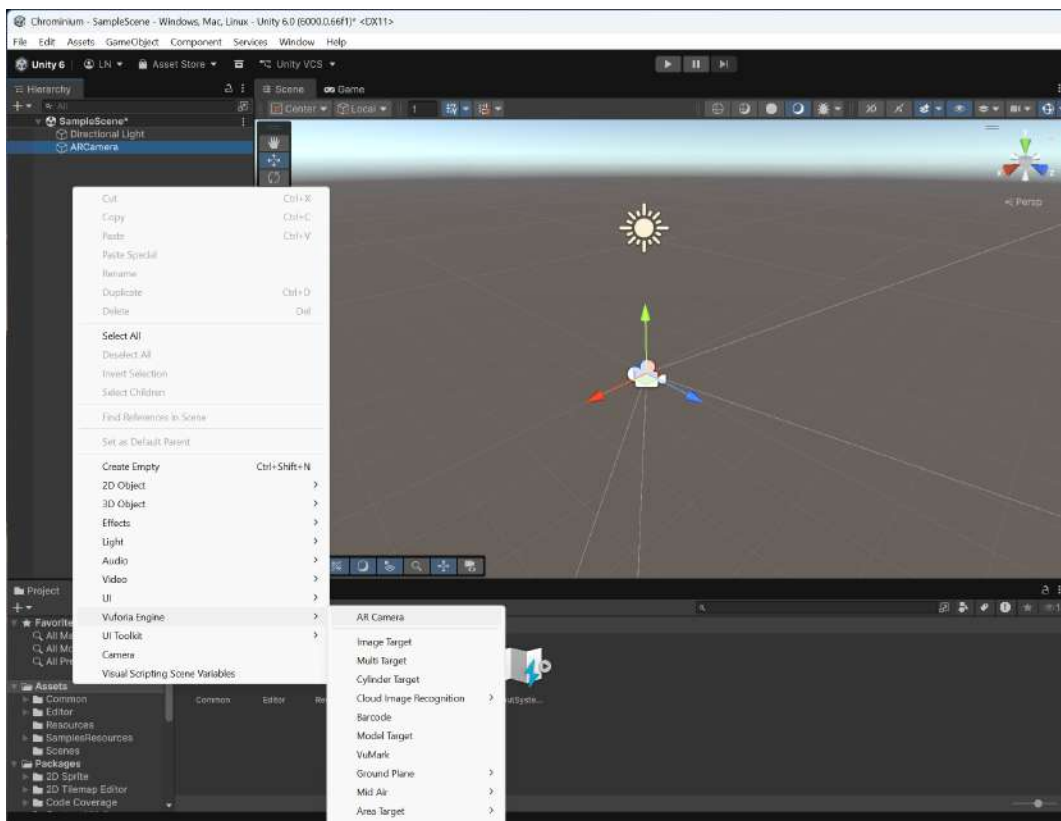


Figura 60: Validación de AR Camera en la jerarquía.

7.67. Compilación y generación de APK

Esta sección resume el flujo de build para Android, asegurando que el proyecto tenga permisos de cámara, configuración de API y firma listos para distribución.

7.68. Configuración básica de Android

Se definen los parámetros de plataforma y compatibilidad para garantizar ejecución en dispositivos reales.

- Ajustes iniciales en *Project Settings ¿Player*.
 - Se selecciona *Android* como plataforma objetivo en *Build Settings*.
 - Se valida el *Package Name* con el formato *com.organizacion.proyecto*.
 - Se establece el *Minimum API Level* acorde a la versión objetivo del dispositivo.

7.69. Permisos y compatibilidad de RA

La cámara es esencial para el tracking de Vuforia y debe declararse en Android.

- Permisos requeridos para el funcionamiento.
 - Se habilita el permiso *Camera* en *Player Settings ¿Other Settings*.
 - Se verifica que *Graphics API* incluya *OpenGLES3* o *Vulkan* según el dispositivo objetivo.
 - Se valida que *Scripting Backend* esté en *IL2CPP* para builds de producción.

7.70. Configuración de firma

La firma garantiza integridad y permite instalar la app fuera del editor.

- Preparación de *Keystore*.
 - Se crea o selecciona un *Keystore* en *Publishing Settings*.
 - Se registra el *Key Alias* y su contraseña.
 - Se valida que el *Keystore* sea consistente para futuras actualizaciones.

7.71. Generación del APK

Se ejecuta el build final con la configuración ya validada.

- Compilación final.
 - En *Build Settings*, se selecciona *Build*.
 - Se guarda el *APK* en la carpeta de entregables del proyecto.
 - Se instala en un dispositivo físico para validar tracking, UI y rendimiento.

7.72. Configuración de icono en Android

El icono adaptativo asegura que la app conserve identidad visual en diferentes launchers y formas de recorte del sistema.

- Asignación del ícono de la aplicación y configuración para dispositivos Android.
 - En la ventana *Project Settings*, sección *Player*, se establece el *Default Icon* con el logotipo del Alfa Romeo.

- En la pestaña de configuración de Android, se despliega la sección *Adaptive (API 26) icons*.
- Se asigna un fondo sólido (*Background*) y la imagen con transparencia frontal (*Foreground*) para asegurar que el sistema operativo adapte correctamente los bordes del ícono.

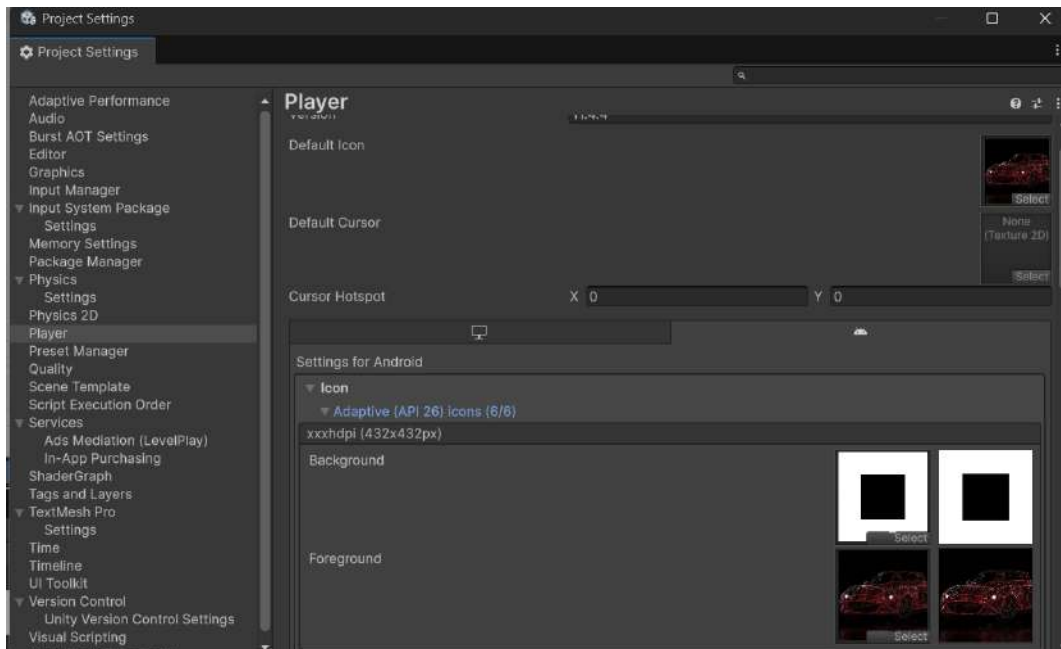


Figura 61: Configuración del icono adaptativo en Android.

8. Análisis de Costos y Viabilidad

→ Presupuesto y Desglose General de Inversión

→ El costo total inicial para el desarrollo y despliegue de la infraestructura de Chromium asciende a \$208,258.31 MXN (\$12,068.54 USD), calculado con una tasa de cambio base de 17.2563 MXN/USD.

→ Los recursos se categorizan estrictamente de la siguiente manera en la tabla de costos:

Concepto	Cantidad	Importe	Categoría
Laptop Lenovo Legion Pro 7i	1	\$55,000.00 MXN (\$3,187.24 USD)	Hardware
Mouse	1	\$120.79 MXN (\$7.00 USD)	Periféricos
Luz (Energía)	1	\$690.25 MXN (\$40.00 USD)	Infraestructura
Smartphone Apple iPhone 8	1	\$2,329.60 MXN (\$135.00 USD)	Hardware

Concepto	Cantidad	Importe	Categoría
Smartphone Motorola G7 Plus	1	\$5,297.68 MXN (\$307.00 USD)	Hardware
Smartphone Samsung S23 Ultra	1	\$11,509.95 MXN (\$667.00 USD)	Hardware
Licencia Unity Pro Anual	1	\$39,862.05 MXN (\$2,310.00 USD)	Software
Licencia SketchFab Pro	1	\$3,106.13 MXN (\$180.00 USD)	Software
Personal Capacitado	1	\$8,628.15 MXN (\$500.00 USD)	Recursos Humanos
Modelos TurboSquid (30)	30	\$51,768.90 MXN (\$3,000.00 USD)	Software
Paq. Hojas Blancas (500)	5	\$1,075.00 MXN (\$62.30 USD)	Material
Licencia Autodesk Maya	1	\$25,884.45 MXN (\$1,500.00 USD)	Software
Internet 350 megas	1	\$845.56 MXN (\$49.00 USD)	Infraestructura
Licencia Apple Developer	1	\$1,708.37 MXN (\$99.00 USD)	Software
Licencia Android Developer	1	\$431.41 MXN (\$25.00 USD)	Software
TOTAL GENERAL		\$208,258.31 MXN (\$12,068.54 USD)	

- Análisis de Depreciación y Costo de Uso de Activos (Fase Inicial de 4 Meses)
- Debido a que el proyecto contempla un periodo crítico de 4 meses para alcanzar su punto de equilibrio, el coste real de los activos fijos se evalúa mediante su depreciación en dicho cuatrimestre:
 - Laptop Lenovo Legion Pro 7i: Valor de \$55,000 MXN con una vida útil de 5 años y depreciación anual del 30 %.
 - Fórmula aplicada: $55,000 \times 0,3 \times (4/12) = 5,500$ MXN de costo de uso.
 - Samsung Galaxy S23 Ultra: Valor de adquisición de \$22,000 MXN con una tasa de depreciación anual del 61 %.
 - Fórmula aplicada: $22,000 \times 0,61 \times (4/12) = 4,473,35$ MXN de costo de uso.
 - Dispositivos Históricos (Apple iPhone 8 y Motorola G7 Plus): Equipos con más de 5 años de antigüedad.
 - Contablemente han superado su vida útil y su valor de depreciación es cero, operando bajo su valor residual de mercado como herramientas de pruebas de compatibilidad de hardware.
- Evaluación de Viabilidad Comercial y Retorno de Inversión (ROI)
- Viabilidad Financiera: Con un costo operativo amortizado, el negocio B2B alcanza su rentabilidad a partir del cuarto mes de despliegue.

- Se proyecta un Retorno de Inversión (ROI) del 43 % a largo plazo.
- Viabilidad Comercial: El modelo resuelve el problema de inventario físico de las concesionarias. Sustituir un local comercial físico tradicional de 500m² en una plaza de alto tráfico por un stand de Realidad Aumentada interactivo reduce los costos fijos drásticamente, justificando el costo del licenciamiento del software.

9. Métricas de éxito (KPIs)

- Técnicas: tasa de error de tracking ¡2 %; uso máximo de RAM ¡1.5 GB; consumo de batería ¡15 % en una sesión de 15 minutos.
- De negocio: tasa de conversión de leads, tiempo de permanencia ¡4 minutos y satisfacción del cliente (CSAT) respecto a la fidelidad visual.

10. Referencias

- Unity Technologies. (2024). Universal Render Pipeline Documentation.
- PTC. (2024). Vuforia Engine Developer Portal & API Reference.
- Poly Haven. (2024). Public Domain HDRIs for Global Illumination.
- SCRUMstudy. (2024). SBOK Guide for Project Management.
- Sketchfab. (n.d.). *2023 Jeep Wrangler Rubicon 392 20th Anniversary* [3D model]. <https://sketchfab.com/3d-models/2023-jeep-wrangler-rubicon-392-20th-anniversary-7247a74596224c3dad219836430279c>
- Sketchfab. (n.d.). *2014 Abarth 500 14-16V* [3D model]. <https://sketchfab.com/3d-models/2014-abarth-500-14-16v-a7fe3d6fa0a44c83a62f21853256d166>
- Sketchfab. (n.d.). *2016 Alfa Romeo Giulia Quadrifoglio* [3D model]. <https://sketchfab.com/3d-models/2016-alfa-romeo-giulia-quadrifoglio-8985b52ac8a84aaf90ccaa5669697001>
- Behance. (s. f.). *Alfa Romeo Giulia GTA - Oleg Markaryan*. Behance. https://www.behance.net/gallery/166847989/Alfa-Romeo-Giulia-GTA?locale=es_ES
- Pin by José Matheus on Carros — Jeep wrangler, Jeep art, Jeep drawing. (s. f.). Pinterest. <https://mx.pinterest.com/pin/979110775222748981/>
- Debutify. (s. f.). *Fiat 500 Abarth Fine Line*. Recal Decals. <https://www.recaldecals.com/products/fiat-abarth>
- sound fx effects free. (2022, 6 diciembre). *success - Sound Effect* [Video]. YouTube. <https://www.youtube.com/watch?v=n4qnDaSidJs>

11. Image Targets utilizados



Figura 62: Image Target: Logo Fiat 500 Abarth.



Figura 63: Image Target: Logo Alfa Romeo Giulia Quadrifoglio.



Figura 64: Image Target: Logo Jeep Wrangler.

12. Consulta y Descarga del repositorio del proyecto en GitHub

<https://github.com/Luis8885nm/CHROMINIUM.git>